Artificial Intelligence: A Mathematical and Engineering Perspective

March 2026

Chapter 1

Learning as Function Approximation

1.1 Learning Problems and Data

Artificial intelligence, in its modern form, is largely concerned with the problem of *learning from data*. At a high level, we are given observations of the world and asked to construct a system that can make predictions, decisions, or generate new data.

We formalize this as follows. Let

$$\mathcal{X} \subseteq \mathbb{R}^d$$

denote the *input space* and

$$\mathcal{Y}$$

the *output space*. A dataset is a collection of samples

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathcal{X},\ y_i \in \mathcal{Y}.$$

The goal of learning is to construct a function

$$f : \mathcal{X} \to \mathcal{Y}$$

that captures the relationship between inputs and outputs.

Examples.

- In *regression*, $\mathcal{Y} \subseteq \mathbb{R}$.
- In *classification*, $\mathcal{Y} = \{1, \ldots, K\}$.
- In *generative modeling*, $\mathcal{Y} = \mathcal{X}$, and the goal is to model the data distribution.

While this formulation appears simple, it hides the central challenge of machine learning: we only observe finitely many samples, yet we seek a function that performs well on unseen data.

1.2 Functions, Models, and Representations

A learning algorithm does not search over all possible functions. Instead, it selects a function from a *model class* (or *hypothesis space*)

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

Here:

- θ denotes parameters,
- Θ is the parameter space,
- f_θ is a function determined by θ.

Example (Linear model).

$$f_\theta(x) = \langle w, x \rangle + b, \quad \theta = (w, b).$$

Example (Neural network).

$$f_\theta(x) = W_L \sigma\big(W_{L-1}\sigma(\cdots \sigma(W_1 x))\big),$$

where σ is a nonlinear activation function.

The choice of $\mathcal{H}$ encodes assumptions about the structure of the problem. This is known as *inductive bias*, a concept we will revisit in later chapters.

Representation matters. Two models may represent the same class of functions but behave very differently in practice due to:

- optimization properties,
- numerical stability,
- generalization behavior.

Thus, a model is not just a function class—it is also a *computational and statistical object*.

1.3 Approximation as the Core Paradigm

At its heart, machine learning is an approximation problem.

We assume that there exists an unknown function

$$f^\star : \mathcal{X} \to \mathcal{Y}$$

that governs the data:

$$y = f^\star(x) + \text{noise}.$$

Since $f^\star$ is unknown, we seek an approximation f_θ such that

$$f_\theta \approx f^\star.$$

To quantify this approximation, we introduce a *loss function*

$$\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}.$$

The quality of f is measured by the *risk*

$$R(f) = \mathbb{E}_{(x,y)}\big[\ell(f(x), y)\big].$$

Since the data distribution is unknown, we instead minimize the *empirical risk*

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i).$$

Learning problem.

$$\min_{\theta \in \Theta} \hat{R}_n(f_\theta).$$

This simple optimization problem is the foundation of nearly all modern machine learning algorithms.

1.4 Examples from Regression and Classification

1.4.1 Regression

In regression, the output space is $\mathcal{Y} = \mathbb{R}$.

A common choice of loss is the squared error:

$$\ell(f(x), y) = (f(x) - y)^2.$$

The empirical risk becomes

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} (f(x_i) - y_i)^2.$$

Interpretation.

- Penalizes large deviations strongly
- Leads to analytically tractable solutions for linear models

1.4.2 Classification

In classification, $\mathcal{Y} = \{1, \ldots, K\}$.

Instead of predicting labels directly, models often output scores or probabilities:

$$f(x) \in \mathbb{R}^K.$$

A common loss is the cross-entropy loss:

$$\ell(f(x), y) = -\log p_\theta(y \mid x).$$

Interpretation.

- Encourages correct class probabilities
- Strong connection to maximum likelihood estimation

1.5 From Approximation to Generalization

A central issue arises: minimizing empirical risk does not guarantee good performance on new data.

Key question. Why should a function that fits the training data well also perform well on unseen data?

This question leads to the study of:

- generalization,
- model complexity,
- regularization,
- inductive bias.

These ideas form the theoretical backbone of machine learning and will be developed throughout this book.

1.6 A Unifying Perspective

We summarize the learning process:

1. Choose a model class $\mathcal{H}$
2. Define a loss function ℓ
3. Solve an optimization problem
4. Evaluate on unseen data

Three interacting components:

- **Approximation** (model expressivity)
- **Optimization** (ability to find good parameters)
- **Generalization** (performance on new data)

Modern artificial intelligence, particularly deep learning, succeeds because it balances these three aspects at scale.

1.7 Outlook

This chapter introduced the central viewpoint of this book: learning as function approximation under uncertainty.

In the next chapter, we develop the probabilistic framework underlying learning, connecting empirical risk minimization to statistical inference.

Guiding principle. Throughout this text, we will emphasize:

- intuition grounded in mathematics,
- models as structured approximations,
- and algorithms as computational realizations of these ideas.

Chapter 2

A Probabilistic View of Learning

2.1 Randomness and Data Generation

In Chapter 1, we introduced learning as the problem of approximating an unknown function from data. We now refine this perspective by incorporating *uncertainty*.

Data is not deterministic. Even when a true relationship exists, observations are subject to noise, variability, and incomplete information. A natural way to model this is through probability.

We assume that data is generated according to an unknown probability distribution

$$P \quad \text{on} \quad \mathcal{X} \times \mathcal{Y}.$$

A dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

is modeled as independent samples drawn from P:

$$(x_i, y_i) \sim P.$$

Interpretation.

- The distribution P captures all properties of the data-generating process.
- The dataset is only a finite, noisy observation of this process.

This viewpoint shifts the goal of learning: instead of fitting a dataset, we aim to learn properties of the underlying distribution.

2.2 Conditional Structure and Prediction

A useful decomposition of P is given by

$$P(x, y) = P(y \mid x)P(x).$$

Key object: the conditional distribution $P(y \mid x)$.

Prediction can be understood as estimating this conditional relationship.

Example (Regression).

$$y = f^\star(x) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid x] = 0.$$

Then

$$f^\star(x) = \mathbb{E}[y \mid x].$$

Example (Classification).

$$P(y = k \mid x), \quad k = 1, \ldots, K.$$

Prediction corresponds to choosing the most probable class:

$$\hat{y} = \arg\max_k P(y = k \mid x).$$

Thus, many learning problems reduce to estimating conditional expectations or probabilities.

2.3 Bayes' Theorem and Inference

Bayes' theorem provides a fundamental relationship between conditional probabilities:

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}.$$

Interpretation.

- $P(y)$ is the prior belief about y,
- $P(x \mid y)$ is the likelihood of observing x given y,
- $P(y \mid x)$ is the posterior belief after observing x.

In practice, Bayes' theorem suggests two complementary approaches:

- **Discriminative modeling:** directly model $P(y \mid x)$
- **Generative modeling:** model $P(x \mid y)$ and $P(y)$

Engineering perspective.

- Discriminative models often yield better predictive performance.
- Generative models provide richer structure and can generate data.

2.4 Loss Functions as Statistical Objectives

In Chapter 1, we introduced loss functions as measures of error. From a probabilistic viewpoint, loss functions arise naturally from statistical principles.

Consider a model f_θ that defines a conditional distribution $p_\theta(y \mid x)$.

Negative log-likelihood.

$$\ell(f_\theta(x), y) = -\log p_\theta(y \mid x).$$

Minimizing the empirical risk

$$\frac{1}{n}\sum_{i=1}^{n} -\log p_\theta(y_i \mid x_i)$$

is equivalent to maximizing the likelihood:

$$\max_\theta \prod_{i=1}^{n} p_\theta(y_i \mid x_i).$$

Examples.

- Gaussian noise $\Rightarrow$ squared loss
- Categorical distribution $\Rightarrow$ cross-entropy loss

Thus, many commonly used loss functions are not arbitrary—they correspond to probabilistic assumptions about the data.

2.5 Risk and Expected Loss

The true objective of learning is to minimize the *expected loss* (or *risk*)

$$R(f) = \mathbb{E}_{(x,y)\sim P}\big[\ell(f(x), y)\big].$$

Ideal problem.

$$\min_{f\in\mathcal{H}} R(f).$$

However, P is unknown, so $R(f)$ cannot be computed exactly.

Instead, we approximate it using the empirical risk:

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i).$$

Empirical Risk Minimization (ERM).

$$\min_{f\in\mathcal{H}} \hat{R}_n(f).$$

ERM is the central principle underlying most machine learning algorithms.

2.6 Approximation, Estimation, and Optimization

The probabilistic formulation reveals three distinct sources of error:

1. Approximation error The model class $\mathcal{H}$ may not contain the true function.

2. Estimation error We only observe a finite sample from P.

3. Optimization error We may not find the exact minimizer of the empirical risk.

Insight. Modern machine learning systems are effective because they balance these three errors.

2.7 Conditional Expectation as an Optimal Predictor

A key theoretical result provides insight into the structure of optimal predictors.

Proposition. For squared loss, the function that minimizes the risk is

$$f^\star(x) = \mathbb{E}[y \mid x].$$

Interpretation.

- The optimal prediction is the conditional mean.
- Learning aims to approximate this conditional expectation.

This result explains why regression models focus on estimating averages, even when data is noisy.

2.8 From Probabilities to Algorithms

While the probabilistic framework provides clarity, practical algorithms must operate under constraints:

- limited data,
- high-dimensional inputs,
- computational limitations.

Thus, models are designed not only to be statistically sound, but also:

- computationally efficient,

- numerically stable,
- scalable to large datasets.

This interplay between probability and computation is a defining feature of modern machine learning.

2.9 A Unifying Perspective

We can now refine the learning pipeline:

1. Assume data is generated from an unknown distribution P
2. Choose a model $p_\theta(y \mid x)$ or function f_θ
3. Define a loss derived from probabilistic principles
4. Minimize empirical risk

Key takeaway. Learning is the process of approximating conditional distributions from finite samples.

2.10 Outlook

This chapter introduced the probabilistic foundation of learning:

- data as samples from a distribution,
- prediction as estimating conditional structure,
- loss functions as statistical objectives.

In the next chapter, we study *hypothesis spaces* and *inductive bias*, which determine what kinds of functions can be learned and how generalization becomes possible.

Guiding principle. Probability provides the language of uncertainty, but successful learning requires combining it with structure, computation, and approximation.

Chapter 3

Hypothesis Spaces and Inductive Bias

3.1 Function Classes and Model Families

In previous chapters, we formulated learning as the problem of selecting a function that minimizes empirical risk. However, we have not yet addressed a crucial question:

Which functions are we allowed to choose from?

This question is answered by specifying a *hypothesis space* (or model class)

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

The hypothesis space defines the set of candidate functions that a learning algorithm can represent.

Examples of function classes.

- **Linear models:**
$$f(x) = \langle w, x \rangle + b.$$
- **Polynomial models:**
$$f(x) = \sum_{|\alpha| \le d} c_\alpha x^\alpha.$$
- **Neural networks:**
$$f_\theta(x) = W_L \sigma\big(W_{L-1} \sigma(\cdots \sigma(W_1 x))\big).$$

Each choice of $\mathcal{H}$ imposes structural assumptions about the relationship between inputs and outputs.

Expressivity. A hypothesis space is said to be expressive if it can approximate a wide range of functions.

Example. Neural networks with sufficient width can approximate any continuous function on compact domains (universal approximation property).

However, expressivity alone does not determine performance. A highly expressive model may fit training data perfectly but fail to generalize.

Thus, the choice of $\mathcal{H}$ is not merely about approximation—it is about balancing expressivity, stability, and learnability.

3.2 Inductive Bias and Prior Structure

The hypothesis space introduces an implicit preference for certain functions over others. This preference is known as *inductive bias*.

Definition (informal). Inductive bias is the set of assumptions that allows a learning algorithm to generalize beyond the observed data.

Without inductive bias, learning is impossible: infinitely many functions can interpolate the training data, but only some will generalize well.

Sources of inductive bias.

- **Model class:** Linear models favor linear relationships.
- **Architecture:** Convolutional networks favor locality and translation invariance.
- **Regularization:** Penalizing large parameters favors simpler functions.
- **Optimization:** Gradient-based methods may prefer certain solutions among many.

Connection to prior knowledge. Inductive bias can be viewed as encoding prior knowledge about the problem:

- smoothness,
- sparsity,
- compositional structure,
- invariances.

Engineering perspective. Successful models are not neutral—they are carefully designed to reflect structure in the data.

For example:

- CNNs exploit spatial locality in images,

- Transformers exploit relationships between elements in sequences.

Inductive bias is therefore not a limitation, but a necessary ingredient for learning.

3.3 Bias–Variance Tradeoff (Intuition)

We now examine how the choice of hypothesis space affects generalization.

Consider the expected prediction error at a point x:

$$\mathbb{E}\big[(f(x) - y)^2\big].$$

This error can be conceptually decomposed into three components:

- **Bias:** error due to systematic approximation
- **Variance:** error due to sensitivity to data
- **Noise:** inherent randomness in the data

High-bias models.

- Simple hypothesis spaces (e.g., linear models)
- May underfit the data
- Low variance but large systematic error

High-variance models.

- Highly expressive models (e.g., deep networks)
- Can fit training data closely
- Sensitive to noise and fluctuations

Tradeoff. Increasing model complexity typically:

- decreases bias,
- increases variance.

Insight. Good generalization requires balancing these two effects.

Remark. In modern deep learning, this classical picture is refined: highly overparameterized models can still generalize well due to implicit regularization and optimization effects.

3.4 Choosing Representations

The performance of a learning system depends not only on the function class, but also on how inputs are represented.

Definition. A representation is a transformation

$$\phi : \mathcal{X} \to \mathcal{Z}$$

that maps raw inputs into a feature space.

A model then operates on $\phi(x)$ instead of x:

$$f(x) = g(\phi(x)).$$

Example.

- Polynomial features: $\phi(x) = (x, x^2, \dots)$
- Learned features in neural networks

Key idea. A simple model applied to a rich representation can be more powerful than a complex model applied to raw inputs.

Feature engineering vs representation learning.

- Classical approaches rely on manually designed features
- Deep learning learns representations automatically

Compositional representations. Deep networks build representations hierarchically:

- lower layers capture local patterns,
- higher layers capture abstract structure.

Geometric viewpoint. Representations transform the geometry of the data so that:

- classes become separable,
- structure becomes simpler,
- learning becomes easier.

Thus, choosing a representation is equivalent to choosing a coordinate system in which the learning problem becomes tractable.

3.5 A Unifying Perspective

We can now refine the learning problem:

1. Choose a representation ϕ
2. Choose a hypothesis space $\mathcal{H}$
3. Impose inductive bias through structure and regularization
4. Optimize empirical risk

Key components of learning:

- **Representation** (how data is encoded)
- **Function class** (what can be expressed)
- **Inductive bias** (what is preferred)

Modern deep learning systems unify these elements: they learn representations and functions simultaneously, guided by implicit and explicit biases.

3.6 Outlook

This chapter emphasized that learning is not only about fitting data, but about choosing the right space of functions and representations.

In the next chapter, we study learning as an optimization problem, focusing on how models are trained in practice.

Guiding principle. Generalization arises not from data alone, but from the interaction between data, model structure, and inductive bias.

Chapter 4

Learning as Optimization

4.1 Optimization Formulations of Learning

In previous chapters, we formulated learning as the problem of minimizing empirical risk. We now examine this problem through the lens of optimization.

Recall that a model is parameterized by $\theta \in \Theta$, and learning consists of solving

$$\min_{\theta \in \Theta} \hat{R}_n(\theta) \quad \text{where} \quad \hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Key observation. Learning reduces to minimizing a function over a high-dimensional parameter space.

This formulation unifies a wide range of models:

- Linear regression $\to$ convex quadratic optimization
- Logistic regression $\to$ convex optimization
- Neural networks $\to$ highly nonconvex optimization

Structure of the objective.

- Sum of losses over data points
- Often smooth (differentiable)
- High-dimensional and possibly nonconvex

Engineering constraint. Exact minimization is rarely possible. Instead, we aim to find parameters that are *good enough* within limited computational time.

Thus, optimization is not merely a mathematical tool—it determines what models are practically usable.

4.2 Gradient-Based Methods

Most modern learning algorithms rely on gradient-based optimization.

Gradient descent. Given a differentiable objective $L(\theta)$, the update rule is

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

where $\eta > 0$ is the learning rate.

Interpretation.

- The gradient $\nabla L(\theta)$ points in the direction of steepest increase
- Moving in the negative gradient direction decreases the loss

Geometric view. Gradient descent follows the local geometry of the loss surface, moving toward regions of lower value.

Choice of learning rate.

- Too small $\rightarrow$ slow convergence
- Too large $\rightarrow$ instability or divergence

Example (quadratic loss). For a convex quadratic function, gradient descent converges to the global minimum under suitable conditions.

Limitation. Computing the full gradient requires summing over all data points:

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell(f_\theta(x_i), y_i).$$

For large datasets, this becomes computationally expensive.

4.3 Stochastic Optimization

To address the computational cost of full gradients, we use stochastic approximations.

Stochastic Gradient Descent (SGD). At each step, sample a data point (or mini-batch) and compute an approximate gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla \ell(f_\theta(x_{i_t}), y_{i_t}).$$

Mini-batch SGD.

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell(f_\theta(x_i), y_i).$$

Key properties.

- Lower computational cost per iteration
- Introduces noise into the optimization process

Insight. The noise in SGD is not purely detrimental—it can help escape poor local minima and improve generalization.

Tradeoff.

- Small batch size $\to$ noisy but cheap updates
- Large batch size $\to$ stable but expensive updates

Practical methods.

- Momentum: accelerates convergence by smoothing updates
- Adaptive methods (e.g., Adam): adjust learning rates per parameter

Stochastic optimization is the backbone of modern deep learning.

4.4 Geometry of Loss Landscapes

To understand optimization behavior, it is useful to consider the geometry of the loss function.

Loss landscape. The function

$$L(\theta)$$

defines a surface over parameter space.

Critical points.

- Local minima: $\nabla L(\theta) = 0$, L is locally minimal
- Saddle points: $\nabla L(\theta) = 0$, but not minima

Convex vs nonconvex.

- Convex problems: all local minima are global
- Nonconvex problems: multiple local minima and saddle points

Neural networks give rise to highly nonconvex loss landscapes, yet gradient-based methods often succeed in practice.

Flat vs sharp minima.

- Flat minima: loss changes slowly around the solution
- Sharp minima: loss changes rapidly

Observation. Solutions found by SGD tend to lie in flatter regions, which are often associated with better generalization.

High-dimensional effects. In high dimensions:

- Saddle points are more common than poor local minima
- Many directions may have near-zero curvature

Geometric intuition. Optimization can be viewed as navigating a complex landscape shaped by:

- model architecture,
- data distribution,
- parameterization.

4.5 Optimization and Generalization

Optimization does more than minimize training loss—it influences generalization.

Key idea. Different optimization algorithms may converge to different solutions, even if they achieve similar training error.

Implicit bias. Gradient-based methods exhibit a preference for certain solutions among many possible minimizers.

Example.

- In overparameterized linear models, gradient descent often finds minimum-norm solutions

Implication. Optimization is part of the modeling process—it implicitly regularizes the learned function.

4.6 A Unifying Perspective

We can now refine the learning process:

1. Define a loss function based on data and model
2. Formulate an optimization problem
3. Apply gradient-based methods (often stochastic)
4. Obtain a solution shaped by both the objective and the optimization dynamics

Three interacting aspects:

- **Objective** (what is minimized)
- **Algorithm** (how it is minimized)
- **Geometry** (structure of the loss landscape)

Modern machine learning succeeds not only because good objectives are defined, but because efficient optimization methods can exploit their structure.

4.7 Outlook

This chapter introduced optimization as a central component of learning:

- learning as minimizing empirical risk,
- gradient-based methods as the primary tool,
- stochasticity as both a necessity and an advantage.

In the next chapter, we study generalization more carefully, addressing the fundamental question of why models that fit data well can still perform well on unseen examples.

Guiding principle. Learning is not only about what we optimize, but also about how we optimize it.

Chapter 5

Generalization and Overfitting

5.1 Training and Test Performance

In previous chapters, we formulated learning as minimizing empirical risk and studied how optimization algorithms find solutions. However, a central question remains:

Why should a model that fits the training data well perform well on unseen data?

To address this, we distinguish between two types of performance.

Training error.

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

Test error (generalization error).

$$R(f) = \mathbb{E}_{(x,y)\sim P}\big[\ell(f(x), y)\big].$$

Key distinction.

- Training error measures fit to observed data
- Test error measures performance on new data

A model generalizes well if its training error is close to its test error.

Generalization gap.

$$R(f) - \hat{R}_n(f).$$

Insight. Minimizing training error alone is not sufficient. The goal of learning is to minimize *test error*, which is not directly observable.

Practical approach. We approximate test performance using a *validation set*, separate from the training data.

5.2 Overfitting and Underfitting

The relationship between training and test error leads to two fundamental failure modes.

Underfitting.

- Model is too simple to capture the structure of the data
- High training error
- High test error

Overfitting.

- Model fits training data too closely, including noise
- Very low training error
- High test error

Illustration. A highly flexible model may interpolate all training points but behave erratically between them.

Tradeoff. As model complexity increases:

- training error typically decreases,
- test error may decrease initially, then increase.

Classical picture. There exists an optimal level of complexity that balances underfitting and overfitting.

Remark (modern perspective). In deep learning, models often operate far beyond this classical regime, achieving low training error without necessarily suffering from poor generalization. This phenomenon will be revisited later.

5.3 Model Capacity and Complexity

To understand overfitting, we must quantify the expressive power of a model.

Model capacity. The ability of a hypothesis space $\mathcal{H}$ to fit a wide range of functions.

Examples.

- Linear models: low capacity
- High-degree polynomials: high capacity
- Deep neural networks: very high capacity

Qualitative effects of capacity.

- Low capacity $\rightarrow$ high bias, low variance
- High capacity $\rightarrow$ low bias, high variance

Complexity measures (informal).

- Number of parameters
- Depth and width of networks
- Norms of parameters

Important insight. Capacity is not determined solely by the number of parameters. The structure of the model and the learning algorithm also play a crucial role.

Example. Two neural networks with the same number of parameters may behave very differently depending on:

- architecture,
- initialization,
- optimization dynamics.

Thus, capacity should be viewed as a combination of representation, parameterization, and training procedure.

5.4 Informal Generalization Principles

Although a complete theory of generalization in modern machine learning is still evolving, several guiding principles are widely observed in practice.

1. Simpler models tend to generalize better.

This principle is often referred to as *Occam's razor*: among models that fit the data, simpler ones are preferred.

Simplicity can take different forms:

- fewer parameters,
- smaller norms,

- smoother functions.

2. Regularization improves generalization.

Regularization introduces additional constraints or penalties that favor simpler solutions.

Examples include:

- weight decay,
- dropout,
- early stopping.

3. More data improves generalization.

As the sample size increases:

- empirical risk better approximates true risk,
- estimation error decreases.

4. Inductive bias is essential.

Generalization depends on assumptions built into the model:

- smoothness,
- invariance,
- compositional structure.

5. Optimization influences generalization.

The choice of optimization algorithm can affect which solution is found, even when multiple solutions achieve low training error.

This phenomenon is known as *implicit regularization*.

6. Noise can be beneficial.

Stochasticity in training (e.g., SGD) can help avoid overfitting by preventing convergence to overly complex solutions.

Summary insight. Generalization is not determined by a single factor, but by the interaction between:

- model capacity,
- data,
- inductive bias,
- optimization.

5.5 A Unifying Perspective

We can now reinterpret learning as the problem of balancing two objectives:

- fitting the training data,
- maintaining the ability to generalize.

Core tension.

$$\text{Fit the data} \quad \text{vs} \quad \text{avoid overfitting.}$$

Practical strategy.

1. Choose an appropriate model class
2. Control capacity through regularization
3. Use validation to guide model selection
4. Monitor training and test performance

Modern viewpoint. Deep learning systems operate in regimes where classical intuitions are only partially valid, yet the underlying principles of inductive bias and structure remain essential.

5.6 Outlook

This chapter introduced generalization as a central challenge in learning:

- distinguishing training and test performance,
- understanding overfitting and underfitting,
- relating model capacity to generalization.

In the next chapter, we study regularization methods in greater detail, focusing on how to control model complexity in practice.

Guiding principle. A good model is not one that fits the data perfectly, but one that captures the underlying structure in a way that extends beyond the observed examples.

Chapter 6

Regularization and Structure

6.1 Norm-Based Regularization

In previous chapters, we saw that highly expressive models can fit training data extremely well, yet risk overfitting. Regularization provides a systematic way to control model complexity and improve generalization.

Basic idea. Augment the empirical risk with a penalty on the parameters:

$$\min_{\theta \in \Theta} \hat{R}_n(\theta) + \lambda\, \Omega(\theta),$$

where:

- $\Omega(\theta)$ measures the complexity of the model,
- $\lambda > 0$ controls the strength of regularization.

Common choices of penalties.

- ℓ_2 **regularization (weight decay):**

$$\Omega(\theta) = \|\theta\|_2^2.$$

- ℓ_1 **regularization:**

$$\Omega(\theta) = \|\theta\|_1.$$

Interpretation.

- ℓ_2 regularization favors small, distributed parameters
- ℓ_1 regularization promotes sparsity

Geometric viewpoint. Regularization restricts the solution to lie in a region of parameter space with controlled norm.

Statistical viewpoint. Norm penalties can be interpreted as imposing prior distributions on parameters:

- $\ell_2 \to$ Gaussian prior
- $\ell_1 \to$ Laplace prior

Practical effect. Regularization reduces variance and improves stability, often at the cost of a small increase in bias.

6.2 Constraints and Penalization

Regularization can be formulated in two equivalent ways:

- **Penalized form:**

$$\min_{\theta} \ \hat{R}_n(\theta) + \lambda \Omega(\theta),$$

- **Constrained form:**

$$\min_{\theta} \ \hat{R}_n(\theta) \quad \text{subject to} \quad \Omega(\theta) \leq C.$$

Insight. These formulations are closely related: for appropriate choices of λ and C, they yield similar solutions.

Examples.

- Ridge regression: ℓ_2 penalty
- Lasso: ℓ_1 penalty

Interpretation. The constraint limits the effective search space, preventing overly complex solutions.

Dual perspective. Penalization trades off between data fit and model simplicity, while constraints enforce a strict bound on complexity.

Engineering perspective. Penalized formulations are often easier to optimize, while constrained formulations provide clearer geometric intuition.

6.3 Implicit Regularization

Not all regularization is explicit. In many modern learning systems, the optimization algorithm itself induces a preference for certain solutions.

Definition (informal). Implicit regularization refers to the tendency of an algorithm to favor specific solutions even in the absence of explicit penalties.

Example (linear models). In overparameterized linear regression, gradient descent converges to the minimum-norm solution among all interpolating solutions.

Example (deep learning).

- SGD tends to find solutions with particular structure
- Different optimizers can lead to different generalization behavior

Sources of implicit regularization.

- Initialization
- Optimization dynamics
- Stochasticity (e.g., mini-batching)

Key insight. Even when a model has enough capacity to perfectly fit the data, the training procedure determines *which* solution is found.

Implication. Understanding generalization requires analyzing not only the model and loss, but also the optimization process.

6.4 Early Stopping and Practical Methods

Regularization can also be achieved through practical training strategies.

Early stopping. Instead of minimizing training loss completely, we stop training when validation performance begins to degrade.

Interpretation.

- Early iterations capture large-scale structure
- Later iterations may fit noise

Connection to regularization. Early stopping behaves similarly to ℓ_2 regularization in many settings.

Other practical methods.

- **Dropout:** randomly deactivate units during training

- **Data augmentation:** expand dataset through transformations
- **Noise injection:** add noise to inputs or parameters

Interpretation.

- Dropout encourages redundancy and robustness
- Data augmentation encodes invariances
- Noise smooths the learned function

Engineering perspective. These methods are often simple to implement yet highly effective, especially in deep learning systems.

6.5 Structure as Regularization

Beyond explicit penalties and training tricks, model structure itself acts as a powerful form of regularization.

Examples.

- Convolutional layers enforce locality
- Residual connections stabilize deep networks
- Attention mechanisms capture relational structure

Key idea. Structure restricts the class of functions that can be represented, thereby guiding learning toward meaningful solutions.

Insight. In modern machine learning, architecture design is often more important than explicit regularization.

6.6 A Unifying Perspective

Regularization can be understood as controlling the effective complexity of the learned function.

Forms of regularization.

- Explicit (penalties, constraints)
- Implicit (optimization dynamics)
- Structural (model architecture)

Unified objective.

$$\text{Fit the data} \quad + \quad \text{control complexity.}$$

Key components.

- Model class
- Regularization mechanism
- Optimization algorithm

These elements interact to determine both training performance and generalization.

6.7 Outlook

This chapter introduced regularization as a central tool for controlling model complexity:

- norm-based penalties,
- constraints and penalization,
- implicit effects of optimization,
- practical techniques such as early stopping and dropout.

In the next part of the book, we turn to concrete supervised learning models, beginning with linear regression and least squares.

Guiding principle. Successful learning requires not only fitting data, but shaping the solution space so that meaningful patterns emerge.

Chapter 7

Linear Regression and Least Squares

7.1 Linear Models and Projections

Linear regression is one of the simplest and most fundamental models in machine learning. Despite its simplicity, it illustrates many key ideas that extend to more complex models.

Model. We consider functions of the form

$$f(x) = \langle w, x \rangle + b,$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, we aim to find parameters (w, b) that best fit the data.

Matrix formulation. Let

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_n^\top & - \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

For simplicity, we incorporate the bias into the parameter vector by augmenting x with a constant component. Then the model becomes

$$f(x) = \langle w, x \rangle,$$

and the prediction vector is

$$\hat{y} = Xw.$$

Geometric interpretation. The columns of X span a subspace of $\mathbb{R}^n$. Linear regression seeks a vector $\hat{y}$ in this subspace that is closest to y.

Thus, linear regression can be viewed as an *orthogonal projection* problem.

7.2 Least Squares as Optimization

A natural choice of loss is the squared error:

$$\ell(f(x_i), y_i) = (f(x_i) - y_i)^2.$$

Objective.

$$\min_w \frac{1}{n}\|Xw - y\|_2^2.$$

This is a convex quadratic optimization problem.

Optimality condition. Setting the gradient to zero yields the normal equations:

$$X^\top X w = X^\top y.$$

Solution (when $X^\top X$ is invertible).

$$w^\star = (X^\top X)^{-1} X^\top y.$$

Projection interpretation. The fitted values $\hat{y} = Xw^\star$ satisfy

$$y - \hat{y} \perp \operatorname{span}(X),$$

meaning the residual is orthogonal to the column space of X.

Insight. The least squares solution is the best approximation of y within the model class in terms of Euclidean distance.

Overparameterized case. If $X^\top X$ is not invertible (e.g., $d > n$), there are infinitely many solutions. In this case, one often selects the minimum-norm solution:

$$w^\star = X^\top (XX^\top)^{-1} y.$$

This solution is closely related to implicit regularization, as discussed in previous chapters.

7.3 Conditioning and Numerical Stability

While the closed-form solution provides insight, practical computation depends on numerical properties of the matrix $X^\top X$.

Conditioning. The condition number of $X^\top X$ measures sensitivity to perturbations:

$$\kappa(X^\top X) = \frac{\lambda_{\max}}{\lambda_{\min}},$$

where $\lambda_{\max}$ and $\lambda_{\min}$ are the largest and smallest eigenvalues.

Ill-conditioning.

- Occurs when $\lambda_{\min}$ is very small
- Leads to unstable solutions
- Small changes in data can produce large changes in w

Causes.

- Highly correlated features
- Redundant or nearly redundant inputs

Practical consequences.

- Numerical instability in inversion
- Poor generalization

Remedies.

- Feature scaling and normalization
- Removing redundant features
- Regularization (discussed next)

Algorithmic note. In practice, direct inversion is often avoided in favor of more stable methods such as QR decomposition or singular value decomposition (SVD).

7.4 Ridge Regression

Ridge regression introduces ℓ_2 regularization into the least squares problem.

Objective.

$$\min_w \|Xw - y\|_2^2 + \lambda \|w\|_2^2, \quad \lambda > 0.$$

Solution.

$$w^\star = (X^\top X + \lambda I)^{-1} X^\top y.$$

Interpretation.

- Adds stability by shifting eigenvalues of $X^\top X$
- Reduces sensitivity to noise
- Controls the magnitude of parameters

Geometric view. Ridge regression shrinks the solution toward the origin, balancing data fit and parameter norm.

Bias–variance tradeoff.

- Increases bias slightly
- Reduces variance significantly

Connection to conditioning. The matrix $X^\top X + \lambda I$ is always invertible for $\lambda > 0$, improving numerical stability.

Bayesian interpretation. Ridge regression corresponds to assuming a Gaussian prior on w.

7.5 A Unifying Perspective

Linear regression illustrates several central themes of machine learning:

- Learning as projection onto a function space
- Optimization as solving structured problems
- Regularization as control of complexity and stability

Three perspectives.

- **Geometric:** projection onto a subspace
- **Algebraic:** solving normal equations
- **Statistical:** estimating conditional expectation

These perspectives extend naturally to more complex models, including neural networks.

7.6 Outlook

This chapter introduced linear regression as a foundational model:

- least squares formulation,
- geometric interpretation,
- numerical considerations,
- regularization via ridge regression.

In the next chapter, we extend these ideas to classification through logistic regression, where probabilistic modeling plays a central role.

Guiding principle. Simple models often reveal deep structure. Understanding them provides insight into more complex learning systems.

Chapter 8

Logistic Regression and Classification

8.1 Probabilistic Interpretation

In contrast to regression, where outputs are real-valued, classification problems involve predicting discrete labels. A natural approach is to model the *probability* of each class given the input.

Binary classification. Let $\mathcal{Y} = \{0, 1\}$. Instead of predicting labels directly, we model

$$P(y = 1 \mid x).$$

Logistic model. We assume

$$P(y = 1 \mid x) = \sigma(\langle w, x \rangle),$$

where σ is the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Properties of the sigmoid.

- Maps $\mathbb{R}$ to $(0, 1)$
- Smooth and monotonic
- Interpretable as probability

Log-odds interpretation. The model implies

$$\log \frac{P(y = 1 \mid x)}{P(y = 0 \mid x)} = \langle w, x \rangle.$$

Thus, logistic regression models the log-odds as a linear function of the input.

Multiclass extension. For K classes, we use the softmax function:

$$P(y = k \mid x) = \frac{\exp(\langle w_k, x \rangle)}{\sum_{j=1}^{K} \exp(\langle w_j, x \rangle)}.$$

This formulation ensures that probabilities are positive and sum to one.

8.2 Cross-Entropy Loss

To train the model, we need a loss function that reflects probabilistic accuracy.

Binary cross-entropy loss.

$$\ell(f(x), y) = -\big(y \log p(x) + (1 - y) \log(1 - p(x))\big),$$

where $p(x) = P(y = 1 \mid x)$.

Multiclass cross-entropy.

$$\ell(f(x), y) = -\log P(y \mid x).$$

Interpretation.

- Penalizes confident but incorrect predictions heavily
- Encourages calibrated probabilities

Geometric insight. Cross-entropy measures the discrepancy between the predicted distribution and the true label distribution.

Practical advantage. Unlike squared loss, cross-entropy leads to well-behaved gradients for classification tasks.

8.3 Maximum Likelihood Perspective

Logistic regression can be derived from the principle of maximum likelihood.

Likelihood function.

$$L(w) = \prod_{i=1}^{n} P(y_i \mid x_i; w).$$

Log-likelihood.

$$\ell(w) = \sum_{i=1}^{n} \log P(y_i \mid x_i; w).$$

Maximizing the log-likelihood is equivalent to minimizing the empirical cross-entropy loss:

$$\min_w \; -\sum_{i=1}^{n} \log P(y_i \mid x_i; w).$$

Binary case. Substituting the sigmoid model yields a convex optimization problem.

Gradient. For binary classification,

$$\nabla_w \ell(w) = \sum_{i=1}^{n} (\sigma(\langle w, x_i \rangle) - y_i) x_i.$$

Insight.

- The gradient reflects prediction error weighted by input features
- Training adjusts parameters to reduce discrepancies between predicted probabilities and labels

Regularization. As in linear regression, one often adds an ℓ_2 penalty:

$$\min_w \; -\sum_{i=1}^{n} \log P(y_i \mid x_i; w) + \lambda \|w\|_2^2.$$

Interpretation. This corresponds to maximum a posteriori (MAP) estimation under a Gaussian prior.

8.4 Decision Boundaries

Although logistic regression models probabilities, it ultimately produces decisions.

Binary decision rule.

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid x) \geq \frac{1}{2}, \\ 0 & \text{otherwise.} \end{cases}$$

Equivalent condition.

$$\langle w, x \rangle \geq 0.$$

Decision boundary. The set of points satisfying

$$\langle w, x \rangle = 0$$

defines a hyperplane in $\mathbb{R}^d$.

Geometric interpretation.

- The input space is divided into two half-spaces
- The normal vector w determines orientation

Multiclass case. Decision boundaries are determined by comparing linear scores:

$$\langle w_k, x \rangle = \langle w_j, x \rangle.$$

Limitation. Logistic regression produces *linear decision boundaries*. It cannot capture nonlinear structure without feature transformations.

Extension. Nonlinear decision boundaries can be obtained by:

- feature mappings,
- kernel methods,
- neural networks.

8.5 A Unifying Perspective

Logistic regression connects several key ideas:

- **Probabilistic modeling:** predicting conditional probabilities
- **Optimization:** minimizing cross-entropy loss
- **Geometry:** linear decision boundaries

Comparison with linear regression.

- Linear regression predicts values
- Logistic regression predicts probabilities

Key insight. Classification can be viewed as modeling uncertainty about discrete outcomes.

8.6 Outlook

This chapter introduced logistic regression as a fundamental classification model:

- probabilistic interpretation,
- cross-entropy loss,
- maximum likelihood estimation,
- geometric structure of decision boundaries.

In the next chapter, we extend linear models using feature maps and kernels, enabling them to capture nonlinear relationships.

Guiding principle. Effective classification requires both expressive models and appropriate probabilistic formulations.

Chapter 9

Feature Maps and Nonlinear Models

9.1 Basis Expansions

Linear models are simple and computationally efficient, but their expressivity is limited: they can only represent linear relationships in the input space. A natural way to overcome this limitation is to transform the inputs before applying a linear model.

Feature map. A feature map is a function

$$\phi : \mathcal{X} \to \mathbb{R}^m,$$

which maps the original input into a new feature space.

We then define a model of the form

$$f(x) = \langle w, \phi(x) \rangle.$$

Example (polynomial features). For $x \in \mathbb{R}$,

$$\phi(x) = (1, x, x^2, \ldots, x^d).$$

Example (interaction features). For $x \in \mathbb{R}^d$, include terms such as $x_i x_j$ to capture interactions.

Key idea. A linear model in feature space corresponds to a nonlinear model in the original input space.

Interpretation. Basis expansions allow us to approximate complex functions by combining simpler building blocks.

Connection to approximation theory. Polynomials, Fourier bases, and other function systems can approximate a wide range of functions when combined linearly.

9.2 Linear Models in Feature Space

Once a feature map is defined, learning proceeds as before:

$$\min_w \frac{1}{n} \sum_{i=1}^{n} \ell(\langle w, \phi(x_i) \rangle, y_i).$$

Matrix formulation. Let $\Phi \in \mathbb{R}^{n \times m}$ be the feature matrix:

$$\Phi = \begin{bmatrix} - & \phi(x_1)^\top & - \\ - & \phi(x_2)^\top & - \\ & \vdots & \\ - & \phi(x_n)^\top & - \end{bmatrix}.$$

Then predictions are given by

$$\hat{y} = \Phi w.$$

Insight. All linear regression and classification methods extend directly to feature space by replacing X with Φ.

Expressivity vs complexity.

- Increasing feature dimension m increases expressivity
- But also increases risk of overfitting and computational cost

Geometric viewpoint. The data is embedded into a higher-dimensional space where it may become linearly separable.

Example. Points that are not separable by a line in $\mathbb{R}^2$ may become separable by a hyperplane after mapping into $\mathbb{R}^3$.

9.3 The Kernel Idea (Intuition)

A challenge arises when the feature space is very high-dimensional or even infinite-dimensional.

Observation. Many algorithms depend on inner products between feature vectors:

$$\langle \phi(x), \phi(x') \rangle.$$

Kernel function. A kernel is a function

$$k(x, x') = \langle \phi(x), \phi(x') \rangle,$$

which computes this inner product without explicitly evaluating $\phi(x)$.

Key idea (kernel trick). We can work implicitly in a high-dimensional feature space by using $k(x, x')$ directly.

Examples of kernels.

- Polynomial kernel:
$$k(x, x') = (\langle x, x' \rangle + c)^d$$
- Gaussian (RBF) kernel:
$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

Interpretation.

- Kernels measure similarity between inputs
- They define an implicit feature space

Practical advantage.

- Avoid explicit high-dimensional computations
- Enable flexible nonlinear models

Limitation.

- Computational cost scales with number of data points
- Less scalable than modern deep learning approaches

9.4 Expressivity through Features

Feature maps determine what functions a model can represent.

Key question. How does the choice of ϕ affect the expressive power of the model?

Insight. A linear model in a sufficiently rich feature space can approximate highly complex functions.

Tradeoff.

- Rich features $\rightarrow$ high expressivity
- But increased risk of overfitting

Design principles for features.

- Encode domain knowledge (e.g., invariances)
- Capture relevant structure
- Maintain computational tractability

From feature engineering to representation learning.

- Classical methods rely on manually designed features
- Deep learning learns features automatically from data

Compositionality. Complex features can be built by composing simpler transformations:

$$\phi(x) = \phi_L(\phi_{L-1}(\cdots \phi_1(x))).$$

This idea forms the foundation of deep neural networks.

Geometric perspective. Feature maps reshape the geometry of the data:

- clusters become separable,
- nonlinear structures become linear,
- distances and similarities are redefined.

9.5 A Unifying Perspective

Feature maps provide a bridge between linear models and nonlinear behavior.

Key components.

- Feature map ϕ
- Linear model in feature space
- Optimization over parameters

Two paradigms.

- **Fixed features:** designed manually (e.g., kernels)
- **Learned features:** optimized jointly with the model (deep learning)

Core insight. Nonlinearity in machine learning often arises not from the model itself, but from the representation of the data.

9.6 Outlook

This chapter introduced feature maps as a way to extend linear models:

- basis expansions for nonlinear modeling,
- linear learning in transformed spaces,
- kernel methods for implicit high-dimensional features,

- the role of representation in expressivity.

In the next chapter, we study support vector machines, which combine geometric insights with kernel methods to produce powerful classifiers.

Guiding principle. The choice of representation often determines the success of a learning system as much as the choice of model.

Chapter 10

Support Vector Machines

10.1 Margin Maximization

In previous chapters, we studied linear classifiers and their extensions via feature maps. Support Vector Machines (SVMs) refine these ideas by introducing a geometric principle: *margin maximization*.

Binary classification setting. Let $y_i \in \{-1, +1\}$ and consider a linear classifier

$$f(x) = \langle w, x \rangle + b.$$

A point is correctly classified if

$$y_i(\langle w, x_i \rangle + b) > 0.$$

Margin. The (functional) margin of a data point is defined as

$$y_i(\langle w, x_i \rangle + b).$$

Geometric margin. The distance of a point to the decision boundary is

$$\frac{y_i(\langle w, x_i \rangle + b)}{\|w\|}.$$

Goal. Find a classifier that maximizes the minimum margin over all training points.

Optimization problem.

$$\max_{w,b} \min_i \frac{y_i(\langle w, x_i \rangle + b)}{\|w\|}.$$

Equivalent formulation. By fixing the scale, we obtain

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(\langle w, x_i\rangle + b) \geq 1 \quad \forall i.$$

Interpretation.

- Maximizing margin $\leftrightarrow$ minimizing $\|w\|$
- Encourages robust separation between classes

Support vectors. Only points that lie closest to the decision boundary (i.e., those with equality in the constraint) influence the solution.

Insight. SVMs focus on the most informative points, rather than all training data.

10.2 Hard and Soft Margins

The formulation above assumes that the data is linearly separable. In practice, this is rarely the case.

Hard-margin SVM.

- Assumes perfect separability
- Enforces constraints:

$$y_i(\langle w, x_i\rangle + b) \geq 1$$

Limitation. Sensitive to noise and outliers; may have no feasible solution.

Soft-margin SVM. Introduce slack variables $\xi_i \geq 0$:

$$y_i(\langle w, x_i\rangle + b) \geq 1 - \xi_i.$$

Objective.

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i,$$

where $C > 0$ controls the tradeoff between margin size and classification error.

Interpretation.

- ξ_i measures violation of the margin constraint
- Larger $C \to$ penalizes errors more strongly
- Smaller $C \to$ allows more flexibility

Connection to loss functions. The soft-margin formulation corresponds to minimizing a *hinge loss*:

$$\ell(x_i, y_i) = \max\big(0, 1 - y_i f(x_i)\big).$$

Insight. SVMs combine margin maximization with robustness to non-separable data.

10.3 Dual Formulation (Conceptual)

The SVM optimization problem admits an alternative representation known as the *dual formulation*.

Key idea. Instead of optimizing over w, we introduce variables α_i associated with each data point.

Dual problem (informal).

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle,$$

subject to constraints on α_i.

Important observations.

- The solution depends only on inner products $\langle x_i, x_j \rangle$
- Only support vectors have nonzero α_i

Reconstruction of w.

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i.$$

Insight. The classifier is expressed as a weighted combination of training points.

Practical significance.

- Enables the use of kernel methods
- Reveals sparsity in the solution

10.4 Kernel Methods Revisited

The dual formulation allows us to extend SVMs to nonlinear classification.

Kernel substitution. Replace inner products with a kernel function:

$$\langle x_i, x_j \rangle \;\rightarrow\; k(x_i, x_j).$$

Resulting classifier.

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i k(x_i, x) + b.$$

Interpretation.

- Implicitly maps data into a feature space

- Constructs nonlinear decision boundaries in the original space

Examples.

- Polynomial kernel
- Gaussian (RBF) kernel

Geometric view. Data is transformed into a space where it becomes linearly separable, and a maximum-margin separator is found there.

Advantages.

- Flexible nonlinear modeling
- Strong theoretical guarantees

Limitations.

- Computational cost grows with dataset size
- Less scalable than modern deep learning approaches

10.5 A Unifying Perspective

Support Vector Machines integrate several core ideas:

- **Geometry:** margin maximization
- **Optimization:** convex quadratic programming
- **Representation:** feature maps and kernels

Key insights.

- Good classifiers maximize separation between classes
- Only a subset of data points determines the solution
- Nonlinearity can be introduced through kernels

SVMs illustrate how combining geometry and optimization leads to powerful learning algorithms.

10.6 Outlook

This chapter introduced support vector machines as a principled approach to classification:

- margin-based learning,
- soft constraints for real-world data,
- dual formulation and sparsity,
- kernel methods for nonlinear models.

In the next chapter, we return to model evaluation and selection, focusing on how to compare models and choose hyperparameters in practice.

Guiding principle. Effective learning balances expressivity with robustness, often through geometric and optimization principles.

Chapter 11

Model Evaluation and Selection

11.1 Training, Validation, and Testing

In previous chapters, we developed models and optimization methods for learning from data. However, a crucial practical question remains:

How do we evaluate and compare models in a reliable way?

A central challenge is that the true risk

$$R(f) = \mathbb{E}_{(x,y)\sim P}[\ell(f(x), y)]$$

is unknown, since the data-generating distribution P is not accessible.

Data splitting. To approximate generalization performance, we divide the available dataset into three parts:

- **Training set:** used to fit model parameters
- **Validation set:** used to tune hyperparameters
- **Test set:** used to estimate final performance

Workflow.

1. Train model on training set
2. Select hyperparameters using validation set
3. Evaluate final model on test set

Key principle. The test set must not be used during model selection, as this would lead to overly optimistic estimates.

Overfitting to validation data. Repeated tuning can indirectly overfit the validation set. This motivates more robust evaluation strategies, such as cross-validation.

11.2 Cross-Validation

Cross-validation provides a systematic way to estimate model performance using limited data.

k-fold cross-validation.

- Partition the dataset into k subsets (folds)
- For each fold:
  - Train on $k-1$ folds
  - Evaluate on the remaining fold
- Average the results across folds

Cross-validation estimate.

$$\hat{R}_{\mathrm{CV}} = \frac{1}{k}\sum_{j=1}^{k} R_j,$$

where R_j is the validation error on fold j.

Advantages.

- Efficient use of data
- More stable performance estimates

Choice of k.

- Small k (e.g., 5): faster, higher bias
- Large k (e.g., leave-one-out): lower bias, higher variance and cost

Practical use. Cross-validation is widely used for hyperparameter tuning, especially when datasets are small or moderate in size.

11.3 Performance Metrics

The choice of evaluation metric depends on the learning task.

11.3.1 Regression Metrics

- **Mean Squared Error (MSE):**

$$\frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$$

- **Mean Absolute Error (MAE):**

$$\frac{1}{n}\sum_{i=1}^{n}|f(x_i) - y_i|$$

Interpretation.

- MSE penalizes large errors more strongly
- MAE is more robust to outliers

11.3.2 Classification Metrics

Accuracy.

$$\frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

Precision and recall.

- Precision: fraction of predicted positives that are correct
- Recall: fraction of actual positives that are detected

F1 score.

$$\text{F1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

ROC and AUC.

- ROC curve plots true positive rate vs false positive rate
- AUC summarizes overall classification performance

Imbalanced data. Accuracy can be misleading when classes are imbalanced; precision, recall, and AUC provide more informative evaluation.

Probabilistic metrics. For models that output probabilities, cross-entropy loss is often used as an evaluation metric.

11.4 Model Comparison

Selecting a model involves comparing multiple candidates based on validation performance.

Hyperparameters.

- Regularization strength
- Model architecture
- Optimization parameters

Selection procedure.

1. Define a set of candidate models
2. Evaluate each model using validation or cross-validation
3. Select the model with best performance

Model selection vs evaluation.

- Model selection uses validation data
- Final evaluation uses test data

Overfitting in model selection. Testing many models increases the risk of selecting one that performs well by chance.

Practical strategies.

- Use cross-validation
- Limit the search space of hyperparameters
- Prefer simpler models when performance is similar

Tradeoffs.

- Accuracy vs complexity
- Performance vs computational cost

Reproducibility. Reliable model comparison requires consistent data splits and controlled experimental settings.

11.5 A Unifying Perspective

Model evaluation and selection are integral parts of the learning process.

Pipeline.

- Train models on data
- Estimate generalization performance
- Select the best model
- Evaluate final performance

Key challenges.

- Limited data
- Variance in estimates
- Risk of overfitting during selection

Core insight. Reliable evaluation requires separating the roles of training, validation, and testing.

11.6 Outlook

This chapter introduced the principles of model evaluation and selection:

- separating data for unbiased evaluation,
- using cross-validation for robust estimates,
- choosing appropriate performance metrics,
- comparing models systematically.

In the next chapter, we study optimization methods in greater depth, focusing on practical techniques used in large-scale machine learning.

Guiding principle. A model is only as good as the way it is evaluated. Reliable evaluation is essential for meaningful learning.

Chapter 12

Optimization in Practice

12.1 Stochastic Gradient Descent

In Chapter 4, we introduced gradient-based optimization as the primary method for training machine learning models. In practice, however, computing full gradients over large datasets is often computationally infeasible.

Stochastic Gradient Descent (SGD). Instead of computing the full gradient,

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell(f_\theta(x_i), y_i),$$

we approximate it using a single randomly selected data point:

$$\theta_{t+1} = \theta_t - \eta \nabla \ell(f_\theta(x_{i_t}), y_{i_t}).$$

Interpretation.

- Each update is a noisy estimate of the true gradient
- The noise arises from sampling

Key properties.

- Low computational cost per iteration
- Enables training on large datasets
- Introduces stochasticity into optimization

Convergence behavior (informal).

- SGD does not converge exactly to a minimum
- Instead, it fluctuates around regions of low loss

Insight. The stochasticity of SGD can help escape sharp minima and explore the loss landscape more effectively.

12.2 Mini-Batching and Efficiency

In practice, SGD is rarely implemented using single data points. Instead, we use mini-batches.

Mini-batch SGD.

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell(f_\theta(x_i), y_i),$$

where B_t is a subset (batch) of data points.

Advantages.

- Reduces variance of gradient estimates
- Exploits parallel computation (e.g., GPUs)
- More stable updates than pure SGD

Batch size tradeoff.

- Small batch:
  - Noisy updates
  - Better generalization in some cases
- Large batch:
  - More accurate gradients
  - Higher computational cost per step

Efficiency considerations.

- Memory constraints limit batch size
- Larger batches benefit from hardware acceleration

Epochs and iterations.

- One epoch: one pass through the dataset
- Iteration: one parameter update

12.3 Momentum and Adaptive Methods

Basic SGD can be slow or unstable, especially in high-dimensional problems. Several enhancements improve its performance.

12.3.1 Momentum

Update rule.

$$v_{t+1} = \beta v_t + \nabla L(\theta_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}.$$

Interpretation.

- Accumulates past gradients
- Smooths updates and accelerates convergence

Geometric intuition. Momentum helps traverse flat regions quickly and reduces oscillations in directions of high curvature.

12.3.2 Adaptive Methods

Adaptive methods adjust learning rates for individual parameters.

Examples.

- AdaGrad
- RMSProp
- Adam

Key idea.

- Scale updates based on past gradient magnitudes
- Use larger steps for infrequent features
- Use smaller steps for frequently updated parameters

Adam (conceptual form).

- Maintains moving averages of gradients and squared gradients
- Combines momentum and adaptive scaling

Practical observation.

- Adaptive methods often converge faster
- SGD with momentum may generalize better in some settings

12.4 Practical Convergence Behavior

In real-world settings, optimization rarely follows idealized theoretical behavior.

Learning rate scheduling.

- Decrease learning rate over time
- Common strategies:
  - step decay,
  - exponential decay,
  - cosine annealing

Warm-up.

- Start with small learning rates
- Gradually increase to stabilize early training

Plateaus and sharp drops.

- Training loss may decrease slowly, then suddenly improve
- Reflects transitions in the loss landscape

Noise and generalization.

- Noise from SGD can act as implicit regularization
- Helps avoid sharp minima

Stopping criteria.

- Fixed number of epochs
- Convergence of training loss
- Validation-based early stopping

Practical challenges.

- Choosing learning rates
- Tuning hyperparameters
- Balancing speed and stability

Insight. Optimization in practice is as much an art as it is a science, guided by both theoretical principles and empirical experience.

12.5 A Unifying Perspective

Optimization in modern machine learning involves multiple interacting components:

- **Algorithm:** SGD and its variants
- **Hyperparameters:** learning rate, batch size
- **Dynamics:** noise, curvature, landscape structure

Core tradeoffs.

- Speed vs stability
- Accuracy vs computational cost
- Optimization vs generalization

Key insight. The behavior of optimization algorithms shapes not only how fast models learn, but also what solutions they find.

12.6 Outlook

This chapter examined optimization from a practical perspective:

- stochastic gradient methods,
- mini-batching and efficiency,
- momentum and adaptive algorithms,
- real-world convergence behavior.

In the next chapter, we study numerical and statistical stability, focusing on how data preprocessing and conditioning affect learning.

Guiding principle. Effective optimization requires understanding both the algorithm and the structure of the problem it is applied to.

Chapter 13

Numerical and Statistical Stability

13.1 Ill-Conditioning

In previous chapters, we studied optimization methods and their practical behavior. However, the effectiveness of these methods depends critically on the numerical properties of the problem.

Definition (informal). A problem is said to be *ill-conditioned* if small changes in the input lead to large changes in the output.

Example (linear regression). The solution

$$w^\star = (X^\top X)^{-1} X^\top y$$

is sensitive to perturbations when $X^\top X$ is nearly singular.

Condition number. For a positive definite matrix A, the condition number is

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

Interpretation.

- Large $\kappa(A) \to$ ill-conditioned problem
- Small $\kappa(A) \to$ well-conditioned problem

Geometric intuition. Ill-conditioning corresponds to elongated level sets of the loss function, making optimization difficult.

Consequences.

- Slow convergence of gradient methods

- Numerical instability in matrix inversion
- Sensitivity to noise

Sources of ill-conditioning.

- Highly correlated features
- Redundant inputs
- Poor scaling of variables

13.2 Scaling and Normalization

A simple but effective way to improve conditioning is to preprocess the data.

Feature scaling. Transform each feature to have comparable magnitude.

Standardization.

$$x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of feature j.

Normalization. Scale inputs to have bounded norms, e.g.,

$$x \leftarrow \frac{x}{\|x\|}.$$

Effect on optimization.

- Makes gradients more balanced across dimensions
- Improves convergence speed
- Reduces numerical instability

Connection to conditioning. Proper scaling reduces the condition number of the problem.

Normalization in deep learning.

- Batch normalization
- Layer normalization

Insight. Normalization can be viewed as dynamically adjusting the geometry of the optimization problem.

13.3 Sensitivity to Noise

Real-world data is often noisy. A stable learning algorithm should not be overly sensitive to small perturbations.

Noise sources.

- Measurement errors
- Incomplete observations
- Intrinsic randomness

Effect on learning.

- Can lead to overfitting
- Can destabilize parameter estimates

Example (least squares). In ill-conditioned problems, small noise in y can cause large changes in $w^\star$.

Bias–variance perspective.

- High-variance models are more sensitive to noise
- Regularization reduces sensitivity

Stability as robustness. A stable algorithm produces similar outputs when inputs are slightly perturbed.

Algorithmic stability (informal). If removing or modifying a single data point does not significantly change the learned model, the algorithm is considered stable.

Insight. Stability is closely related to generalization: stable algorithms tend to generalize better.

13.4 Robustness Considerations

Beyond numerical stability, we are often concerned with robustness to broader forms of perturbation.

Types of robustness.

- Robustness to noise in data
- Robustness to distribution shifts
- Robustness to adversarial perturbations

Robust loss functions.

- Mean squared error: sensitive to outliers
- Absolute error: more robust

Regularization and robustness. Constraining model complexity often improves robustness.

Data augmentation. Introducing variations in training data can improve robustness to transformations.

Adversarial perspective (informal). Small, carefully chosen perturbations can significantly affect model predictions, especially in high-dimensional settings.

Insight. Robustness requires both stable models and appropriate assumptions about the data.

13.5 A Unifying Perspective

Numerical and statistical stability are closely related:

- **Numerical stability:** sensitivity to computational errors
- **Statistical stability:** sensitivity to data variations

Key factors.

- Conditioning of the problem
- Data preprocessing
- Model complexity
- Regularization

Core insight. Stable learning systems are those that behave predictably under small perturbations, both numerical and statistical.

13.6 Outlook

This chapter examined stability from both numerical and statistical perspectives:

- ill-conditioning and its consequences,
- the role of scaling and normalization,

- sensitivity to noise,
- robustness considerations.

In the next chapter, we transition from classical models to neural networks, motivating deep learning as a natural extension of these ideas.

Guiding principle. Stability is essential for reliable learning: models should not only fit data, but do so in a way that is robust to perturbations and uncertainty.

Chapter 14

From Classical Models to Neural Networks

14.1 Limitations of Shallow Models

In previous chapters, we studied a range of classical models, including linear regression, logistic regression, and kernel methods. These models are powerful and well-understood, yet they exhibit fundamental limitations.

Shallow models. We refer to models of the form

$$f(x) = \langle w, \phi(x) \rangle,$$

where ϕ is either fixed or involves only a single layer of transformation.

Key limitation: representation. The success of these models depends critically on the choice of features $\phi(x)$.

Challenges.

- Designing effective features requires domain expertise
- Fixed feature maps may fail to capture complex structure
- High-dimensional feature spaces can be computationally expensive

Expressivity limitations.

- Linear models produce linear decision boundaries
- Kernel methods provide flexibility, but scale poorly with data size

Curse of dimensionality. Explicit feature expansions often require exponentially many features to capture complex patterns.

Insight. Classical models separate representation and learning: features are designed first, and then a model is fitted. This separation can limit their effectiveness.

14.2 Compositional Structure of Functions

Many real-world functions exhibit a *compositional structure*.

Definition (informal). A function is compositional if it can be written as a composition of simpler functions:

$$f(x) = f_L(f_{L-1}(\cdots f_1(x))).$$

Examples.

- Images: edges $\to$ shapes $\to$ objects
- Language: characters $\to$ words $\to$ sentences

Hierarchical structure. Each layer captures patterns at a different level of abstraction.

Efficiency of composition. Compositional representations can describe complex functions using relatively few parameters.

Contrast with shallow models. A shallow model may require exponentially many units to represent a function that can be expressed compactly using multiple layers.

Insight. Depth allows reuse of intermediate computations, leading to more efficient representations.

14.3 Motivation for Deep Learning

Deep learning arises from the idea of learning both representations and functions through composition.

Neural networks. A neural network with L layers has the form

$$f(x) = f_L(f_{L-1}(\cdots f_1(x))),$$

where each layer is typically an affine transformation followed by a nonlinearity.

Key advantages.

- **Automatic feature learning:** Features are learned from data rather than manually designed.
- **Expressivity:** Deep networks can represent highly complex functions.
- **Scalability:** Efficient training with large datasets and modern hardware.

Universal approximation (informal). Neural networks can approximate a wide class of functions, given sufficient capacity.

Practical success. Deep learning has achieved strong performance in:

- computer vision,
- natural language processing,
- speech recognition,
- generative modeling.

Key shift. Instead of designing features, we design *architectures* that can learn features automatically.

14.4 Transition to Learned Representations

The transition from classical models to deep learning involves a change in how representations are constructed.

Classical approach.

- Feature map ϕ is fixed
- Model learns parameters w

Deep learning approach.

- Feature map ϕ_θ is parameterized
- Representation and model are learned jointly

Model structure.

$$f(x) = g_\theta(\phi_\theta(x)),$$

where both ϕ_θ and g_θ depend on parameters.

Hierarchical representations.

- Early layers: local or low-level features
- Intermediate layers: patterns and structures
- Final layers: task-specific representations

Geometric viewpoint. Deep networks transform data through successive nonlinear mappings, reshaping its geometry to make learning easier.

Optimization perspective. Learning involves optimizing a highly nonconvex objective over all layers simultaneously.

Regularization through structure. Architectural choices impose inductive biases that guide learning:

- convolutional layers encode locality,

- attention mechanisms encode relationships,
- residual connections stabilize training.

Insight. Deep learning unifies representation, modeling, and optimization into a single framework.

14.5 A Unifying Perspective

We can now reinterpret the evolution of models:

- Classical models: fixed representations + simple models
- Kernel methods: implicit high-dimensional representations
- Deep learning: learned hierarchical representations

Core idea. The power of modern machine learning lies in learning representations that make the problem easier.

Key components.

- Representation (features)
- Function (mapping)
- Optimization (learning)

Deep learning integrates these components in a unified and scalable way.

14.6 Outlook

This chapter motivated the transition from classical models to neural networks:

- limitations of shallow models,
- importance of compositional structure,
- advantages of deep architectures,
- shift toward learned representations.

In the next chapter, we begin our study of deep learning by examining neural networks as compositional function approximators.

Guiding principle. Complex problems often require structured representations, and depth provides a natural way to build them.

Chapter 15

Neural Networks as Compositional Models

15.1 Layered Architectures

Neural networks are structured as compositions of simple transformations organized into layers. This layered architecture is the defining feature that enables them to model complex relationships.

Basic structure. A neural network with L layers defines a function

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x),$$

where each layer has the form

$$f_\ell(z) = \sigma(W_\ell z + b_\ell).$$

Components.

- W_ℓ: weight matrix
- b_ℓ: bias vector
- σ: nonlinear activation function

Forward propagation. Given an input x, the network computes intermediate representations:

$$z^{(1)} = f_1(x), \quad z^{(2)} = f_2(z^{(1)}), \quad \ldots, \quad z^{(L)} = f(x).$$

Interpretation.

- Each layer transforms the representation of the data

- The final output depends on successive refinements

Common activation functions.

- ReLU: $\sigma(z) = \max(0, z)$
- Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
- Tanh: $\sigma(z) = \tanh(z)$

Role of nonlinearity. Without nonlinear activation functions, a composition of linear layers reduces to a single linear transformation.

Insight. Nonlinearity enables neural networks to represent complex, non-linear relationships.

15.2 Expressivity and Depth

A central question is how the architecture of a neural network influences its expressive power.

Width vs depth.

- **Width:** number of units per layer
- **Depth:** number of layers

Shallow vs deep networks.

- Shallow networks: few layers, possibly very wide
- Deep networks: many layers, often narrower

Key observation. Depth allows functions to be represented more efficiently.

Example (compositional functions). Functions with hierarchical structure can be represented compactly by deep networks, but may require exponentially many parameters in shallow networks.

Geometric intuition. Each layer reshapes the input space, gradually transforming complex structures into simpler ones.

Piecewise linearity. Networks with ReLU activations represent piecewise linear functions with potentially many regions.

Insight. Depth increases the number of linear regions exponentially, enhancing expressivity.

Tradeoff.

- Greater depth $\rightarrow$ higher expressivity
- But also more challenging optimization

15.3 Universal Approximation (Insight)

A fundamental theoretical result explains why neural networks are powerful approximators.

Universal approximation (informal). A feedforward neural network with a single hidden layer and a suitable activation function can approximate any continuous function on a compact domain, given sufficient width.

Interpretation.

- Neural networks are highly expressive
- Even shallow networks can approximate complex functions

Limitation.

- The theorem does not specify how many units are required
- Practical learning may be difficult

Role of depth. While shallow networks are theoretically sufficient, deep networks often provide more efficient representations.

Insight. Universal approximation guarantees existence, but not efficiency or learnability.

Connection to practice. Deep architectures are preferred because they:

- require fewer parameters,
- are easier to train in many cases,
- align with compositional structure in data.

15.4 Representation Learning

One of the defining features of neural networks is their ability to learn representations automatically.

Learned features. Each layer produces a representation

$$z^{(\ell)} = f_\ell(z^{(\ell-1)}),$$

which encodes information about the input.

Hierarchy of representations.

- Early layers: local or low-level features
- Intermediate layers: patterns and structures

- Final layers: task-specific features

Contrast with classical methods.

- Classical models use fixed feature maps
- Neural networks learn features from data

Geometric viewpoint. Each layer transforms the geometry of the data, making it progressively easier to separate or model.

Compositional learning. Features are built by combining simpler features across layers.

Interpretation. Learning involves discovering both:

- how to represent the data,
- how to map representations to outputs.

Insight. Representation learning is a key reason for the success of deep learning across diverse domains.

15.5 A Unifying Perspective

Neural networks can be understood through three complementary viewpoints:

- **Functional:** compositions of nonlinear functions
- **Geometric:** transformations of data representations
- **Computational:** layered processing architectures

Core components.

- Parameters (W_ℓ, b_ℓ)
- Nonlinear activations
- Compositional structure

Key insight. The power of neural networks lies not only in their expressivity, but in their ability to learn structured representations through composition.

15.6 Outlook

This chapter introduced neural networks as compositional models:

- layered architectures,
- the role of depth in expressivity,
- universal approximation as a theoretical foundation,
- representation learning as a central principle.

In the next chapter, we study how neural networks are trained efficiently using backpropagation and automatic differentiation.

Guiding principle. Complex functions can often be understood as compositions of simpler ones, and neural networks provide a framework for learning such compositions from data.

Chapter 16

Backpropagation and Automatic Differentiation

16.1 Computational Graphs

Neural networks are compositions of functions. To compute gradients efficiently, it is useful to represent these computations explicitly.

Computational graph. A computational graph is a directed acyclic graph (DAG) that represents how a function is built from simpler operations.

Nodes and edges.

- Nodes represent variables or intermediate values
- Edges represent functional dependencies

Example. Consider a simple function

$$z = (x_1 + x_2) \cdot x_3.$$

We can represent this as a graph:

- $a = x_1 + x_2$
- $z = a \cdot x_3$

Forward pass. We evaluate the graph from inputs to outputs, computing all intermediate values.

Insight. A neural network is a large computational graph, where each layer corresponds to a set of operations.

16.2 Chain Rule and Reverse-Mode Differentiation

To train a neural network, we must compute gradients of a loss function with respect to all parameters.

Chain rule. For a composition of functions,

$$f(x) = f_L(f_{L-1}(\cdots f_1(x))),$$

the derivative is obtained by repeated application of the chain rule.

Local derivatives. Each node in the computational graph contributes a local derivative.

Reverse-mode differentiation. Gradients are computed by propagating sensitivities from the output backward through the graph.

Basic principle. If a variable z depends on x through intermediate variables, then

$$\frac{\partial L}{\partial x} = \sum_{\text{paths}} \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

Backpropagation. This process of propagating gradients backward through the network is known as backpropagation.

Example. For $z = a \cdot x_3$ and $a = x_1 + x_2$,

$$\frac{\partial z}{\partial x_1} = \frac{\partial z}{\partial a} \cdot \frac{\partial a}{\partial x_1}.$$

Insight. Backpropagation systematically applies the chain rule across the computational graph.

16.3 Efficient Gradient Computation

A naive computation of gradients would be computationally expensive, especially for deep networks.

Key idea. Reuse intermediate computations from the forward pass.

Forward pass.

- Compute all intermediate values
- Store them for reuse

Backward pass.

- Compute gradients in reverse order

- Reuse stored intermediate values

Complexity. Backpropagation computes gradients with a cost comparable to evaluating the function itself.

Gradient flow. Gradients propagate layer by layer:

$$\frac{\partial L}{\partial W_\ell} \quad \text{depends on} \quad \frac{\partial L}{\partial z^{(\ell)}}.$$

Shared subcomputations. The computational graph ensures that repeated operations are computed only once.

Insight. Backpropagation is efficient because it avoids redundant computation and exploits the structure of the network.

16.4 Implementation Perspectives

Modern machine learning frameworks implement backpropagation using automatic differentiation.

Automatic differentiation (AD).

- Computes exact derivatives (up to numerical precision)
- Works for arbitrary compositions of differentiable operations

Two modes of AD.

- Forward mode
- Reverse mode (used in deep learning)

Why reverse mode? Efficient when the function has many parameters and a single output (typical in learning problems).

Practical workflow.

1. Define the model as a computational graph
2. Perform a forward pass to compute loss
3. Use automatic differentiation to compute gradients
4. Update parameters using an optimization algorithm

Memory considerations.

- Intermediate values must be stored for backward pass
- Large models require careful memory management

Numerical issues.

- Vanishing and exploding gradients
- Finite precision arithmetic

Insight. Automatic differentiation abstracts the complexity of gradient computation, allowing practitioners to focus on model design.

16.5 A Unifying Perspective

Backpropagation connects several core ideas:

- **Mathematics:** chain rule for derivatives
- **Computation:** efficient reuse of intermediate values
- **Algorithms:** scalable training of deep networks

Key components.

- Computational graph
- Forward pass
- Backward pass

Core insight. Backpropagation enables efficient training of complex models by systematically applying the chain rule.

16.6 Outlook

This chapter introduced backpropagation and automatic differentiation:

- computational graphs as a representation of models,
- reverse-mode differentiation for efficient gradients,
- practical implementation via automatic differentiation.

In the next chapter, we study the challenges of training deep networks, including initialization and gradient-related issues.

Guiding principle. Efficient computation of gradients is the key that makes modern deep learning possible.

Chapter 17

Training Deep Networks

17.1 Initialization Strategies

Training deep neural networks involves optimizing highly nonconvex objectives in high-dimensional parameter spaces. The choice of initialization plays a crucial role in determining whether optimization succeeds.

Random initialization. Parameters are typically initialized randomly:

$$W_\ell^{(0)} \sim \text{distribution}, \quad b_\ell^{(0)} = 0 \text{ or small values.}$$

Why not initialize to zero?

- Symmetry: all units would behave identically
- No meaningful learning would occur

Variance preservation. A key principle is to choose initialization so that activations neither vanish nor explode as they propagate through layers.

Example (Xavier/Glorot initialization).

$$\mathrm{Var}(W_{ij}) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Example (He initialization).

$$\mathrm{Var}(W_{ij}) \approx \frac{2}{n_{\text{in}}}.$$

Insight. Proper initialization maintains stable signal propagation across layers.

Effect on training.

- Good initialization $\rightarrow$ faster convergence
- Poor initialization $\rightarrow$ slow or failed training

17.2 Vanishing and Exploding Gradients

A major difficulty in training deep networks arises from the behavior of gradients.

Gradient propagation. Backpropagation involves repeated multiplication by weight matrices and derivatives:

$$\frac{\partial L}{\partial z^{(\ell)}} = \frac{\partial L}{\partial z^{(\ell+1)}} W_{\ell+1}^{\top} \cdot \sigma'(z^{(\ell)}).$$

Vanishing gradients.

- Gradients shrink exponentially with depth
- Early layers learn very slowly

Exploding gradients.

- Gradients grow exponentially
- Leads to instability and numerical issues

Causes.

- Repeated multiplication of small or large values
- Saturating activation functions (e.g., sigmoid)

Mitigation strategies.

- Careful initialization
- Use of non-saturating activations (e.g., ReLU)
- Normalization methods
- Gradient clipping

Insight. Stable gradient flow is essential for effective learning in deep networks.

17.3 Optimization Challenges

Deep networks introduce optimization challenges beyond those encountered in classical models.

Nonconvexity. The loss function has many local minima and saddle points.

High dimensionality. The parameter space can contain millions or billions of variables.

Saddle points.

- More common than poor local minima
- Can slow down optimization

Flat vs sharp regions.

- Flat regions: slow progress
- Sharp regions: instability

Ill-conditioning. Different directions may have very different curvature, making optimization difficult.

Practical strategies.

- Use stochastic optimization (SGD)
- Apply momentum or adaptive methods
- Normalize inputs and activations

Insight. Despite theoretical complexity, simple optimization methods often work well in practice due to structure in real-world problems.

17.4 Empirical Training Behavior

Training deep networks often exhibits characteristic patterns that are not fully captured by theory.

Training dynamics.

- Rapid initial decrease in loss
- Slower convergence in later stages

Generalization behavior.

- Training loss may approach zero
- Test performance may still be good

Double descent (informal). Increasing model size can first increase and then decrease test error.

Role of overparameterization.

- Large models are easier to optimize
- Many solutions achieve low training error

Implicit regularization. Optimization dynamics favor certain solutions, often with good generalization properties.

Learning curves.

- Plot training and validation error over time
- Used to diagnose underfitting and overfitting

Practical observations.

- Larger datasets improve performance
- Deeper models can capture more structure
- Careful tuning is often required

Insight. Empirical behavior provides important guidance when theory is incomplete.

17.5 A Unifying Perspective

Training deep networks involves several interacting components:

- **Initialization:** sets the starting point
- **Gradient flow:** determines signal propagation
- **Optimization:** navigates the loss landscape
- **Data and model size:** influence generalization

Core challenge. Ensuring stable and efficient training in high-dimensional, nonconvex settings.

Key insight. Successful training relies on aligning architecture, initialization, and optimization.

17.6 Outlook

This chapter examined the practical challenges of training deep networks:

- initialization strategies,
- vanishing and exploding gradients,
- optimization difficulties,
- empirical training behavior.

In the next chapter, we study activation functions and architectural design choices, which further influence the behavior of deep networks.

Guiding principle. Training deep networks requires balancing mathematical principles with empirical insight.

Chapter 18

Activation Functions and Architectures

18.1 Nonlinear Activations

Nonlinear activation functions are a fundamental component of neural networks. They enable networks to represent complex, non-linear relationships that cannot be captured by linear transformations alone.

Role of activation functions. Each layer in a neural network applies a transformation of the form

$$z \mapsto \sigma(Wz + b),$$

where σ is applied elementwise.

Why nonlinearity is necessary. If σ is linear, then a composition of layers reduces to a single linear transformation:

$$W_L W_{L-1} \cdots W_1 x,$$

eliminating the benefit of depth.

Common activation functions.

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Tanh:**

$$\sigma(z) = \tanh(z)$$

- **ReLU (Rectified Linear Unit):**

$$\sigma(z) = \max(0, z)$$

Properties.

- Sigmoid and tanh are smooth but can saturate
- ReLU is simple and avoids saturation in the positive region

Variants.

- Leaky ReLU
- ELU
- GELU

Gradient behavior.

- Saturating activations $\rightarrow$ vanishing gradients
- Piecewise linear activations $\rightarrow$ stable gradients

Insight. The choice of activation function strongly influences both expressivity and optimization dynamics.

18.2 Architectural Design Patterns

Beyond individual layers, the overall structure of a neural network plays a crucial role in its performance.

Fully connected networks. Each layer connects to all units in the previous layer.

Convolutional architectures.

- Exploit spatial locality
- Share parameters across locations

Residual connections.

- Add identity mappings between layers:

$$z^{(\ell+1)} = z^{(\ell)} + F(z^{(\ell)})$$

- Improve gradient flow and stability

Normalization layers.

- Batch normalization
- Layer normalization

Modular design. Modern architectures are often composed of repeated blocks (e.g., convolutional blocks, transformer blocks).

Insight. Architectural design encodes inductive biases that guide learning.

18.3 Depth versus Width

The structure of a neural network is characterized by its depth and width.

Width. Number of units in each layer.

Depth. Number of layers.

Shallow wide networks.

- Can approximate many functions
- May require large numbers of parameters

Deep networks.

- Capture hierarchical structure
- Often more parameter-efficient

Expressivity comparison. Certain functions can be represented efficiently by deep networks but require exponentially large shallow networks.

Optimization considerations.

- Deeper networks are harder to train
- Techniques such as residual connections mitigate this

Insight. Depth provides a natural way to encode compositional structure.

18.4 Expressivity Considerations

The expressive power of a neural network depends on its architecture and activation functions.

Expressivity. The ability of a model to approximate a wide range of functions.

Factors affecting expressivity.

- Depth and width
- Choice of activation function
- Connectivity structure

Piecewise linear networks. ReLU networks represent functions composed of many linear regions.

Geometric viewpoint. Each layer partitions the input space, increasing the complexity of the representation.

Tradeoffs.

- High expressivity $\rightarrow$ better approximation
- But increased risk of overfitting and optimization difficulty

Implicit constraints. Architectures restrict the class of functions that can be represented, providing inductive bias.

Insight. Expressivity alone is not sufficient; effective learning requires a balance between representation power and trainability.

18.5 A Unifying Perspective

Activation functions and architectures jointly determine:

- how information flows through the network,
- how complex functions can be represented,
- how easily the model can be trained.

Core components.

- Nonlinear activations
- Layer connectivity
- Architectural patterns

Key insight. The design of neural architectures is a central aspect of modern machine learning, combining mathematical structure with empirical performance.

18.6 Outlook

This chapter examined the role of activation functions and architectures:

- nonlinear activations enabling expressive models,
- architectural patterns shaping learning,
- tradeoffs between depth and width,
- factors influencing expressivity.

In the next chapter, we study normalization methods, which play a key role in stabilizing and accelerating training.

Guiding principle. The effectiveness of deep learning systems depends not only on their size, but on how they are structured.

Chapter 19

Normalization Methods

19.1 Batch Normalization

Training deep neural networks can be sensitive to the scale and distribution of activations. Normalization methods address this issue by standardizing intermediate representations.

Batch normalization (BatchNorm). Given activations z in a layer, BatchNorm transforms them as follows:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}},$$

where μ_B and σ_B^2 are the mean and variance computed over a mini-batch.

Affine transformation. After normalization, a learnable transformation is applied:

$$y_i = \gamma \hat{z}_i + \beta,$$

where γ and β are parameters.

Purpose.

- Stabilize the distribution of activations
- Improve gradient flow
- Accelerate training

Interpretation. Batch normalization reduces sensitivity to the scale of parameters and inputs.

Training vs inference.

- During training: statistics computed from mini-batches
- During inference: running averages are used

Practical effect.

- Allows larger learning rates
- Reduces dependence on initialization

19.2 Layer and Instance Normalization

While BatchNorm relies on batch statistics, alternative normalization methods operate independently of the batch.

19.2.1 Layer Normalization

Definition. Layer normalization normalizes across features within a single data point:

$$\hat{z} = \frac{z - \mu_{\text{layer}}}{\sqrt{\sigma^2_{\text{layer}} + \varepsilon}}.$$

Properties.

- Independent of batch size
- Suitable for sequence models (e.g., transformers)

19.2.2 Instance Normalization

Definition. Instance normalization normalizes each sample independently, often across spatial dimensions in vision tasks.

Applications.

- Image generation
- Style transfer

Comparison of methods.

- BatchNorm: uses batch statistics
- LayerNorm: uses feature statistics per sample
- InstanceNorm: uses spatial statistics per sample

Insight. Different normalization methods reflect different assumptions about the structure of the data.

19.3 Stabilization of Training

Normalization methods improve the stability and efficiency of training.

Effect on optimization.

- Smooths the loss landscape
- Reduces sensitivity to parameter scaling
- Improves conditioning

Gradient flow.

- Prevents vanishing and exploding gradients
- Maintains consistent signal propagation

Internal covariate shift (informal). As parameters change during training, the distribution of activations shifts. Normalization reduces this effect.

Regularization effect.

- BatchNorm introduces noise through batch statistics
- Can improve generalization

Learning rate interaction.

- Enables use of larger learning rates
- Leads to faster convergence

Insight. Normalization reshapes the optimization problem, making it easier to solve.

19.4 Theoretical and Practical Insights

Although normalization methods are widely used, their theoretical understanding is still evolving.

Scale invariance. Normalization makes the network less sensitive to scaling of parameters.

Reparameterization. Normalization can be viewed as changing the parameterization of the model, which affects optimization dynamics.

Geometry of optimization. Normalization effectively modifies the geometry of the loss landscape, often making it more well-conditioned.

Empirical observations.

- Improves training speed
- Enhances stability across architectures
- Essential in many modern deep networks

Limitations.

- BatchNorm depends on batch size
- May behave poorly with very small batches

Design considerations.

- Choice of normalization depends on architecture and data
- Often combined with other techniques (e.g., residual connections)

Insight. Normalization is both a practical tool and a conceptual mechanism for improving optimization.

19.5 A Unifying Perspective

Normalization methods can be understood as transformations that:

- standardize representations,
- improve conditioning,
- stabilize gradient flow,
- introduce implicit regularization.

Core components.

- Normalization step
- Learnable scaling and shifting
- Interaction with optimization dynamics

Key insight. Normalization plays a central role in making deep networks trainable at scale.

19.6 Outlook

This chapter examined normalization methods in deep learning:

- batch normalization and its variants,
- alternative normalization strategies,
- their role in stabilizing training,
- theoretical and practical perspectives.

In the next chapter, we study regularization techniques specific to deep learning, including dropout and data augmentation.

Guiding principle. Well-designed transformations of intermediate representations can significantly improve both optimization and generalization.

Chapter 20

Regularization in Deep Learning

20.1 Dropout and Noise Injection

Deep neural networks are highly expressive and can easily overfit training data. Regularization techniques are therefore essential for improving generalization.

Dropout. Dropout is a stochastic regularization method in which units are randomly deactivated during training.

Mechanism. For each unit, a binary mask is sampled:

$$m_i \sim \text{Bernoulli}(p),$$

and activations are modified as

$$\tilde{z}_i = m_i z_i.$$

Interpretation.

- Each iteration trains a different subnetwork
- Prevents co-adaptation of features

Inference. At test time, activations are scaled to account for dropout:

$$z_i \leftarrow p\, z_i.$$

Effect.

- Improves robustness
- Reduces overfitting

Noise injection. More generally, noise can be added to inputs, activations, or parameters:

$$z \leftarrow z + \varepsilon,$$

where ε is random noise.

Insight. Noise encourages the model to learn stable features that are insensitive to small perturbations.

20.2 Weight Decay

Weight decay is one of the most widely used regularization techniques in deep learning.

Objective.

$$\min_{\theta} \ \hat{R}_n(\theta) + \lambda \|\theta\|_2^2.$$

Interpretation.

- Penalizes large parameter values
- Encourages simpler models

Effect on optimization. The update rule becomes

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \nabla \hat{R}_n(\theta_t).$$

Insight. Parameters are gradually shrunk toward zero during training.

Connection to stability. Smaller weights often lead to smoother functions and improved generalization.

Bayesian interpretation. Weight decay corresponds to a Gaussian prior on parameters.

20.3 Data Augmentation

Data augmentation improves generalization by artificially increasing the diversity of the training data.

Basic idea. Generate new training examples by applying transformations:

$$(x, y) \ \rightarrow \ (T(x), y),$$

where T preserves the label.

Examples (vision).

- Rotation, translation, scaling

- Flipping, cropping
- Color perturbations

Examples (other domains).

- Noise injection in inputs
- Synonym replacement in text

Interpretation.

- Encodes invariances (e.g., rotation invariance)
- Expands effective dataset size

Geometric view. Augmentation enlarges the region of input space associated with each label.

Insight. Data augmentation acts as a form of regularization by constraining the function to behave consistently under transformations.

20.4 Implicit Bias of Optimization

Regularization in deep learning is not limited to explicit techniques. Optimization itself introduces a form of implicit bias.

Overparameterization. Deep networks often have more parameters than data points, leading to many solutions with zero training error.

Key question. Which solution does the optimization algorithm select?

Implicit bias. Gradient-based methods tend to favor solutions with particular properties, such as:

- small norms,
- smooth functions,
- certain structural patterns.

Example (linear models). Gradient descent converges to the minimum-norm solution among all interpolating solutions.

Deep learning perspective.

- SGD noise influences the trajectory of optimization
- Different optimizers lead to different solutions

Insight. The training process itself acts as a regularizer.

Interaction with explicit regularization. Implicit and explicit regularization often work together to shape the learned model.

20.5 A Unifying Perspective

Regularization in deep learning takes multiple forms:

- **Explicit:** weight decay, dropout
- **Data-driven:** augmentation
- **Implicit:** optimization dynamics

Core objective.

$$\text{Fit data} \quad + \quad \text{promote generalization.}$$

Key insight. Generalization is shaped not only by the model, but by how it is trained and what data it sees.

20.6 Outlook

This chapter examined regularization techniques specific to deep learning:

- dropout and noise injection,
- weight decay,
- data augmentation,
- implicit bias of optimization.

In the next chapter, we study residual networks and skip connections, which play a key role in enabling very deep architectures.

Guiding principle. Effective regularization arises from combining model structure, data transformations, and optimization dynamics.

Chapter 21

Residual Networks and Skip Connections

21.1 The Degradation Problem

As neural networks become deeper, one might expect their performance to improve due to increased expressivity. However, in practice, deeper networks can be harder to train.

Observation. Increasing depth may lead to higher training error, even though the deeper model can represent the same functions as a shallower one.

Degradation problem.

- Deeper networks perform worse than shallower counterparts
- Not due to overfitting, but due to optimization difficulty

Intuition. In principle, additional layers could learn the identity mapping and preserve performance. However, standard training procedures struggle to discover such solutions.

Connection to gradients.

- Deep compositions amplify vanishing and exploding gradients
- Information may degrade across layers

Insight. Depth increases expressivity, but also introduces optimization challenges that must be addressed through architectural design.

21.2 Residual Learning

Residual networks (ResNets) address the degradation problem by introducing *skip connections*.

Residual block.

$$z^{(\ell+1)} = z^{(\ell)} + F(z^{(\ell)}),$$

where F is a learnable transformation.

Interpretation.

- Instead of learning a mapping $H(z)$, the network learns a residual:

$$F(z) = H(z) - z$$

- The full mapping becomes $H(z) = z + F(z)$

Advantages.

- Easier to learn small perturbations of identity
- Improves gradient flow across layers
- Reduces degradation

Gradient propagation. Gradients can flow directly through the identity connection:

$$\frac{\partial z^{(\ell+1)}}{\partial z^{(\ell)}} = I + \frac{\partial F}{\partial z^{(\ell)}}.$$

Insight. Skip connections provide alternative paths for information and gradients, stabilizing training.

21.3 Dynamical Systems Perspective

Residual networks admit an interpretation as discrete dynamical systems.

Discrete dynamics.

$$z^{(\ell+1)} = z^{(\ell)} + F(z^{(\ell)}).$$

Continuous-time analogy. This resembles a discretization of an ordinary differential equation:

$$\frac{dz(t)}{dt} = F(z(t)).$$

Euler discretization. With step size $\Delta t = 1$,

$$z(t + \Delta t) \approx z(t) + \Delta t\, F(z(t)).$$

Interpretation.

- Layers correspond to time steps
- Network depth corresponds to evolution over time

Stability. Residual connections help maintain stable trajectories in representation space.

Insight. Viewing neural networks as dynamical systems provides a framework for understanding depth, stability, and training dynamics.

21.4 Very Deep Architectures

Residual connections enable the training of extremely deep networks.

Empirical observation.

- Networks with hundreds or thousands of layers can be trained successfully
- Performance improves with depth when using residual connections

Architectural patterns.

- Stacking residual blocks
- Combining with normalization layers
- Using bottleneck structures for efficiency

Scaling behavior.

- Deeper models capture more complex patterns
- Training remains stable due to skip connections

Generalization. Residual networks often generalize well despite large capacity.

Limitations.

- Increased computational cost
- Architectural complexity

Insight. Careful architectural design makes depth practically usable.

21.5 A Unifying Perspective

Residual networks illustrate how architectural innovations can address fundamental challenges in deep learning.

Key components.

- Skip connections
- Residual mappings
- Stable gradient flow

Core ideas.

- Learn perturbations rather than full transformations
- Facilitate optimization through identity mappings
- Interpret networks as dynamical systems

Key insight. Architectural design can fundamentally change the behavior of learning algorithms.

21.6 Outlook

This chapter introduced residual networks and skip connections:

- the degradation problem in deep networks,
- residual learning as a solution,
- connections to dynamical systems,
- enabling very deep architectures.

In the next chapter, we study convolutional neural networks, which exploit spatial structure in data.

Guiding principle. Effective architectures combine mathematical structure with practical considerations to make deep learning feasible at scale.

Chapter 22

Convolutional Neural Networks

22.1 Convolution as Structured Linear Operator

Convolutional neural networks (CNNs) are designed to exploit structure in data, particularly in images and signals. The central operation is convolution, which can be understood as a structured linear transformation.

Discrete convolution. For a one-dimensional input $x \in \mathbb{R}^n$ and a kernel $w \in \mathbb{R}^k$, the convolution is defined as

$$(y * w)_i = \sum_{j=1}^{k} w_j \, x_{i-j}.$$

Two-dimensional convolution. For images, convolution is applied over spatial dimensions:

$$y(i,j) = \sum_{u,v} w(u,v) \, x(i-u, j-v).$$

Linear operator view. Convolution can be written as a matrix-vector multiplication:

$$y = Kx,$$

where K is a structured (Toeplitz or block-Toeplitz) matrix.

Key property: parameter sharing.

- The same kernel is applied across all locations
- Reduces number of parameters

Local connectivity.

- Each output depends only on a local region of the input
- Reflects spatial locality in data

Insight. Convolution imposes structure on linear transformations, making them efficient and suitable for structured data.

22.2 Translation Invariance

A key motivation for convolutional architectures is translation invariance.

Translation equivariance. A convolutional layer satisfies:

$$x(\cdot - \tau) \;\to\; y(\cdot - \tau),$$

meaning that shifting the input results in a corresponding shift in the output.

Interpretation.

- Features detected at one location can also be detected elsewhere
- The model does not depend on absolute position

From equivariance to invariance.

- Equivariance preserves spatial structure
- Invariance removes sensitivity to position

Achieving invariance.

- Pooling operations
- Global averaging

Insight. Translation invariance reflects a key property of visual perception: objects can be recognized regardless of their location.

22.3 Pooling and Hierarchies

CNNs typically include pooling operations that reduce spatial resolution while preserving important features.

Pooling.

- Max pooling: selects the maximum value in a region
- Average pooling: computes the average value

Effect.

- Reduces dimensionality
- Introduces invariance to small transformations

Hierarchical structure.

- Early layers detect simple features (edges, textures)
- Intermediate layers detect patterns (shapes, parts)
- Deeper layers detect complex structures (objects)

Receptive field. The region of the input that influences a given unit grows with depth.

Insight. CNNs build representations hierarchically, combining local features into global patterns.

22.4 Applications in Vision

CNNs have achieved remarkable success in visual tasks.

Image classification.

- Assign a label to an image
- Example: object recognition

Object detection.

- Identify and localize objects within an image

Segmentation.

- Assign a label to each pixel
- Enables detailed understanding of scenes

Generative modeling.

- Image synthesis
- Super-resolution

Architectural examples.

- Convolutional layers + pooling
- Residual connections
- Encoder–decoder structures

Insight. CNNs are effective because they incorporate domain-specific structure into the model.

22.5 A Unifying Perspective

Convolutional neural networks combine several key ideas:

- **Structured linear operators:** convolution
- **Inductive bias:** locality and translation invariance
- **Hierarchy:** multi-scale representation learning

Core components.

- Convolutional layers
- Nonlinear activations
- Pooling operations

Key insight. By incorporating structure into the architecture, CNNs achieve both efficiency and strong generalization.

22.6 Outlook

This chapter introduced convolutional neural networks:

- convolution as a structured linear operator,
- translation equivariance and invariance,
- hierarchical feature learning,
- applications in vision.

In the next chapter, we study sequence models, which extend neural networks to sequential and temporal data.

Guiding principle. Effective models exploit the structure of the data they are designed to process.

Chapter 23

Sequence Models

23.1 Recurrent Neural Networks

Many real-world data types are sequential, including time series, text, and speech. Sequence models aim to capture dependencies across time.

Recurrent neural networks (RNNs). An RNN processes a sequence $(x_1, x_2, \ldots, x_T)$ by maintaining a hidden state h_t that evolves over time:

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b),$$

$$y_t = \psi(W_y h_t + c),$$

where ϕ and ψ are nonlinear functions.

Interpretation.

- h_t summarizes past information
- The same parameters are reused across time steps

Parameter sharing. Unlike feedforward networks, RNNs share weights across time, making them suitable for variable-length sequences.

Unrolling in time. An RNN can be viewed as a deep network with shared parameters:

$$h_1 \to h_2 \to \cdots \to h_T.$$

Insight. RNNs model temporal dependencies by recursively updating a hidden state.

23.2 Gated Architectures (LSTM, GRU)

Standard RNNs struggle to capture long-term dependencies. Gated architectures address this limitation.

23.2.1 Long Short-Term Memory (LSTM)

Core idea. Introduce a memory cell that can preserve information over long time periods.

Gates.

- Input gate: controls new information
- Forget gate: controls what to discard
- Output gate: controls what to expose

Structure (conceptual).

- Cell state evolves with controlled updates
- Gates regulate information flow

Effect.

- Enables learning of long-term dependencies
- Mitigates vanishing gradient problem

23.2.2 Gated Recurrent Unit (GRU)

Simplified architecture.

- Combines input and forget mechanisms
- Fewer parameters than LSTM

Update mechanism.

- Update gate controls memory retention
- Reset gate controls influence of past state

Insight. Gating mechanisms allow the network to dynamically control information flow.

23.3 Long-Term Dependencies

A central challenge in sequence modeling is capturing dependencies across long time spans.

Problem.

- Information from early time steps may be lost
- Gradients may vanish during backpropagation through time

Backpropagation through time (BPTT). Training RNNs involves propagating gradients through many time steps:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}.$$

Gradient behavior.

- Repeated multiplication can lead to vanishing or exploding gradients

Role of gating.

- Provides paths for gradients to flow more easily
- Allows selective memory retention

Practical techniques.

- Gradient clipping
- Truncated backpropagation

Insight. Modeling long-term dependencies requires both architectural and optimization considerations.

23.4 Limitations of Recurrence

Despite their strengths, recurrent architectures have important limitations.

Sequential computation.

- Processing must occur step by step
- Limits parallelization

Difficulty with long-range dependencies.

- Even gated architectures have limits

- Information may degrade over long sequences

Optimization challenges.

- Training can be unstable
- Sensitive to hyperparameters

Memory bottleneck. A fixed-size hidden state must summarize the entire past.

Insight. Recurrence provides a natural framework for sequences, but may not scale effectively to complex long-range dependencies.

23.5 A Unifying Perspective

Sequence models extend neural networks to temporal data:

- **State:** hidden representation evolves over time
- **Dynamics:** recursive updates define temporal behavior
- **Memory:** information is stored in hidden states

Architectural evolution.

- RNN $\rightarrow$ LSTM/GRU $\rightarrow$ attention-based models

Key insight. The design of sequence models revolves around managing information flow across time.

23.6 Outlook

This chapter introduced sequence models:

- recurrent neural networks,
- gated architectures,
- challenges of long-term dependencies,
- limitations of recurrence.

In the next chapter, we study attention mechanisms, which address many of the limitations of recurrent models.

Guiding principle. Effective sequence modeling requires mechanisms for preserving and accessing relevant information over time.

Chapter 24

Attention and Transformers

24.1 Attention Mechanisms

Sequence models based on recurrence compress past information into a fixed-size hidden state. This creates a bottleneck when modeling long sequences.

Core idea of attention. Instead of relying on a single hidden state, attention allows the model to directly access all relevant parts of the input.

Query-key-value framework. Attention is defined in terms of three components:

- Query (q)
- Keys (k_i)
- Values (v_i)

Attention weights.

$$\alpha_i = \frac{\exp(\langle q, k_i \rangle)}{\sum_j \exp(\langle q, k_j \rangle)}.$$

Output.

$$\mathrm{Attn}(q, K, V) = \sum_i \alpha_i v_i.$$

Interpretation.

- The query determines which inputs are relevant
- Weights α_i represent importance

Insight. Attention replaces fixed memory with a flexible, data-dependent mechanism.

24.2 Self-Attention

Self-attention applies the attention mechanism within a single sequence.

Definition. Each element of the sequence attends to all other elements.

Construction. Given inputs $x_1, \ldots, x_T$, define:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

Attention output.

$$y_i = \sum_{j=1}^{T} \alpha_{ij} v_j, \quad \alpha_{ij} = \frac{\exp(\langle q_i, k_j \rangle)}{\sum_l \exp(\langle q_i, k_l \rangle)}.$$

Properties.

- Captures dependencies between all positions
- Fully parallelizable across sequence elements

Positional encoding. Since self-attention is permutation-invariant, positional information must be added explicitly.

Insight. Self-attention provides a flexible mechanism for modeling relationships within sequences.

24.3 Transformer Architecture

The transformer architecture is built entirely from attention mechanisms.

Basic components.

- Multi-head self-attention
- Feedforward layers
- Residual connections
- Normalization layers

Multi-head attention. Multiple attention mechanisms are applied in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, \ldots, H_h) W_O,$$

where each head H_i computes attention with different projections.

Feedforward network.

$$\text{FFN}(x) = \sigma(W_1 x + b_1) W_2 + b_2.$$

Residual structure.

$$x \leftarrow x + \mathrm{Layer}(x),$$

followed by normalization.

Architecture layout.

- Stack of identical blocks
- Each block: attention + feedforward

Encoder–decoder structure.

- Encoder processes input sequence
- Decoder generates output sequence with attention to encoder outputs

Insight. Transformers replace recurrence with attention, enabling efficient modeling of long-range dependencies.

24.4 Scaling and Applications

Transformers have demonstrated remarkable scalability and performance across domains.

Scaling behavior.

- Performance improves with model size, data, and computation
- Enables training of very large models

Parallelization.

- Unlike RNNs, transformers process sequences in parallel
- Efficient use of modern hardware

Applications.

- Natural language processing (translation, text generation)
- Vision (vision transformers)
- Speech processing
- Multimodal learning

Pretraining and fine-tuning.

- Large models trained on massive datasets

- Adapted to specific tasks through fine-tuning

Limitations.

- Quadratic complexity in sequence length
- High computational cost

Insight. Scaling and data availability are key drivers of modern deep learning performance.

24.5 A Unifying Perspective

Attention and transformers unify several ideas:

- **Representation:** dynamic weighting of inputs
- **Computation:** parallel processing of sequences
- **Architecture:** modular and scalable design

Core components.

- Query-key-value attention
- Multi-head mechanisms
- Residual and normalization layers

Key insight. Attention provides a flexible mechanism for modeling interactions, replacing fixed sequential structures.

24.6 Outlook

This chapter introduced attention mechanisms and transformers:

- attention as a flexible memory mechanism,
- self-attention for sequence modeling,
- transformer architectures,
- scaling behavior and applications.

In the next chapter, we study generative models, beginning with probabilistic modeling and latent variable approaches.

Guiding principle. Modern architectures succeed by enabling flexible, scalable interactions between data elements.

Chapter 25

Foundations of Generative Modeling

25.1 Density Estimation and Sampling

Generative modeling aims to learn the underlying distribution of data and generate new samples from it.

Objective. Given data $x \sim p_{\text{data}}$, learn a model $p_\theta(x)$ that approximates the true distribution.

Density estimation.

- Explicit models define a probability density $p_\theta(x)$
- Learning is typically based on maximum likelihood:

$$\max_\theta \ \mathbb{E}_{x \sim p_{\text{data}}}[\log p_\theta(x)].$$

Sampling. Once a model is learned, we generate new samples:

$$x \sim p_\theta(x).$$

Two perspectives.

- **Likelihood-based:** directly model $p_\theta(x)$
- **Implicit:** define a sampling procedure without explicit density

Challenges.

- High-dimensional data
- Complex distributions

- Efficient sampling

Insight. Generative modeling involves both representing distributions and generating samples from them.

25.2 Latent Variable Models

Many generative models introduce latent variables to capture underlying structure.

Latent variable formulation.

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz,$$

where z is a latent variable.

Interpretation.

- z represents hidden factors of variation
- $p(z)$ is a simple prior (e.g., Gaussian)

Generative process.

1. Sample $z \sim p(z)$
2. Generate $x \sim p_\theta(x \mid z)$

Learning challenge. The marginal likelihood involves an intractable integral.

Inference. We introduce an approximate posterior $q_\phi(z \mid x)$.

Insight. Latent variables provide a structured way to model complex data distributions.

25.3 Energy-Based Perspectives

An alternative view models probability distributions through energy functions.

Energy-based model (EBM).

$$p_\theta(x) = \frac{1}{Z(\theta)} \exp(-E_\theta(x)),$$

where $E_\theta(x)$ is an energy function and $Z(\theta)$ is the partition function:

$$Z(\theta) = \int \exp(-E_\theta(x))\, dx.$$

Interpretation.

- Low energy $\rightarrow$ high probability
- High energy $\rightarrow$ low probability

Learning.

- Requires estimating gradients involving $Z(\theta)$
- Often uses sampling-based methods

Connection to physics. Energy-based models resemble distributions in statistical mechanics.

Insight. Energy-based formulations provide a flexible way to define distributions without requiring normalized densities in closed form.

25.4 Evaluation of Generative Models

Evaluating generative models is challenging due to the complexity of high-dimensional distributions.

Likelihood-based evaluation.

- Log-likelihood
- Perplexity (for sequences)

Sample quality.

- Visual inspection (e.g., images)
- Human evaluation (e.g., text)

Divergence measures.

- Kullback–Leibler divergence
- Jensen–Shannon divergence

Distributional metrics.

- Inception Score
- Fréchet Inception Distance (FID)

Challenges.

- Tradeoff between sample quality and diversity
- Difficulty of estimating likelihood in high dimensions

Mode collapse. Some models generate limited diversity, focusing on a subset of possible outputs.

Insight. No single metric fully captures the quality of a generative model.

25.5 A Unifying Perspective

Generative modeling can be understood through multiple complementary frameworks:

- **Probabilistic:** modeling $p(x)$ directly
- **Latent variable:** introducing hidden structure
- **Energy-based:** defining distributions via energy

Core challenges.

- High-dimensional modeling
- Efficient sampling
- Reliable evaluation

Key insight. Generative models aim to capture the structure of data distributions in a way that enables both understanding and synthesis.

25.6 Outlook

This chapter introduced the foundations of generative modeling:

- density estimation and sampling,
- latent variable models,
- energy-based perspectives,
- evaluation of generative models.

In the next chapters, we study specific classes of generative models, including variational autoencoders, generative adversarial networks, and diffusion models.

Guiding principle. Generative modeling seeks not only to predict data, but to understand and reproduce its underlying structure.

Chapter 26

Likelihood-Based Learning with Latent Variables

26.1 Maximum Likelihood Estimation

A central principle in generative modeling is maximum likelihood estimation (MLE). The goal is to choose model parameters that maximize the probability of observed data.

Objective. Given data $\{x_i\}_{i=1}^n$, we maximize:

$$\max_\theta \frac{1}{n} \sum_{i=1}^n \log p_\theta(x_i).$$

Interpretation.

- Encourages the model to assign high probability to observed data
- Equivalent to minimizing the negative log-likelihood

Connection to divergence. MLE minimizes the Kullback–Leibler divergence:

$$\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta).$$

Gradient.

$$\nabla_\theta \log p_\theta(x)$$

can often be computed efficiently for explicit models.

Insight. MLE provides a principled way to learn probabilistic models from data.

26.2 Hidden Variables and Marginalization

Latent variable models introduce hidden variables z to capture underlying structure.

Marginal likelihood.

$$p_\theta(x) = \int p_\theta(x, z)\, dz = \int p_\theta(x \mid z) p(z)\, dz.$$

Log-likelihood.

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz.$$

Interpretation.

- z explains variability in x
- The model integrates over all possible latent explanations

Challenge. The integral over z is often intractable, especially in high dimensions.

Posterior distribution.

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)}.$$

Insight. Learning requires both evaluating the marginal likelihood and reasoning about latent variables.

26.3 Intractability and Approximation

Exact computation of the marginal likelihood is typically infeasible.

Sources of intractability.

- High-dimensional latent space
- Complex likelihood functions

Approximation strategies.

- Sampling-based methods (e.g., Monte Carlo)
- Deterministic approximations
- Variational methods

Monte Carlo approximation.

$$p_\theta(x) \approx \frac{1}{N}\sum_{i=1}^{N} p_\theta(x \mid z_i), \quad z_i \sim p(z).$$

Limitations.

- High variance estimates
- Computationally expensive

Alternative viewpoint. Rather than approximating the integral directly, we approximate the posterior $p_\theta(z \mid x)$.

Insight. Approximation is unavoidable in latent variable models and shapes both theory and practice.

26.4 Variational Principles

Variational methods provide a powerful framework for approximating intractable quantities.

Variational distribution. Introduce a tractable distribution $q_\phi(z \mid x)$ to approximate the true posterior.

Key identity.

$$\log p_\theta(x) = \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] + \mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)).$$

Evidence lower bound (ELBO).

$$\mathcal{L}(\theta,\phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x,z) - \log q_\phi(z \mid x)].$$

Lower bound property.

$$\mathcal{L}(\theta,\phi) \le \log p_\theta(x).$$

Decomposition.

$$\mathcal{L}(\theta,\phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)).$$

Interpretation.

- First term: reconstruction quality
- Second term: regularization toward the prior

Optimization.

- Maximize ELBO with respect to θ and ϕ
- Jointly learn generative and inference models

Insight. Variational methods transform intractable inference into an optimization problem.

26.5 A Unifying Perspective

Likelihood-based learning with latent variables involves several interconnected ideas:

- **MLE:** learning from observed data
- **Latent variables:** modeling hidden structure
- **Approximation:** handling intractable integrals
- **Variational inference:** optimization-based approximation

Core tradeoff.

$$\text{Accuracy of approximation} \quad \text{vs} \quad \text{computational tractability.}$$

Key insight. Introducing latent variables increases modeling power but requires approximate inference techniques.

26.6 Outlook

This chapter introduced likelihood-based learning with latent variables:

- maximum likelihood estimation,
- marginalization over hidden variables,
- challenges of intractability,
- variational principles and the ELBO.

In the next chapter, we apply these ideas to variational autoencoders, a practical and widely used class of generative models.

Guiding principle. Complex probabilistic models require approximate inference, and variational methods provide a principled and scalable solution.

Chapter 27

Variational Autoencoders

27.1 Latent Representations

Variational autoencoders (VAEs) are generative models that combine latent variable modeling with neural networks.

Core idea. Learn a low-dimensional latent representation z that captures the underlying structure of data x.

Generative model.

$$z \sim p(z), \quad x \sim p_\theta(x \mid z),$$

where $p(z)$ is typically chosen as a standard Gaussian.

Latent space.

- Continuous, low-dimensional representation
- Encodes semantic structure of data

Interpretation.

- Nearby points in latent space correspond to similar data
- Interpolation in latent space produces meaningful variations

Goal. Learn both:

- a generative model $p_\theta(x \mid z)$,
- an inference model $q_\phi(z \mid x)$.

Insight. Latent representations provide a structured way to model complex data distributions.

27.2 Evidence Lower Bound (ELBO)

VAEs are trained by maximizing the evidence lower bound introduced in the previous chapter.

ELBO objective.

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)).$$

Interpretation.

- First term: reconstruction accuracy
- Second term: regularization toward the prior

Tradeoff.

- Good reconstruction vs structured latent space

Reparameterization trick. To optimize the expectation, we express z as:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Benefit.

- Enables gradient-based optimization
- Reduces variance of gradient estimates

Insight. The ELBO balances data reconstruction and latent structure, forming the core of VAE training.

27.3 Encoder–Decoder Architectures

VAEs are implemented using neural networks in an encoder–decoder framework.

Encoder.

$$q_\phi(z \mid x)$$

maps input data to latent parameters $(\mu_\phi(x), \sigma_\phi(x))$.

Decoder.

$$p_\theta(x \mid z)$$

maps latent variables back to the data space.

Architecture.

- Encoder network: compresses data into latent representation
- Decoder network: reconstructs data from latent code

Training pipeline.

1. Encode x to (μ, σ)
2. Sample z using reparameterization
3. Decode z to reconstruct x
4. Compute ELBO and update parameters

Sampling. New data is generated by:

- Sampling $z \sim p(z)$
- Computing $x \sim p_\theta(x \mid z)$

Insight. The encoder–decoder structure integrates inference and generation into a unified framework.

27.4 Practical Considerations

Implementing VAEs involves several important design choices.

Choice of likelihood.

- Gaussian likelihood for continuous data
- Bernoulli likelihood for binary data

KL divergence term. Often has a closed form when both $q_\phi(z \mid x)$ and $p(z)$ are Gaussian.

Posterior collapse.

- The model ignores latent variables
- $q_\phi(z \mid x)$ becomes close to $p(z)$

Mitigation strategies.

- KL annealing
- More expressive decoders

Latent space structure.

- Smoothness enables interpolation
- Regularization encourages meaningful representations

Evaluation.

- ELBO as training objective
- Sample quality and diversity

Insight. Practical performance depends on balancing reconstruction, regularization, and model capacity.

27.5 A Unifying Perspective

Variational autoencoders combine several key ideas:

- **Latent variable modeling**
- **Variational inference**
- **Neural network parameterization**

Core components.

- Encoder $q_\phi(z \mid x)$
- Decoder $p_\theta(x \mid z)$
- ELBO objective

Key insight. VAEs provide a scalable method for learning probabilistic generative models with latent structure.

27.6 Outlook

This chapter introduced variational autoencoders:

- latent representations,
- ELBO optimization,
- encoder–decoder architectures,
- practical considerations.

In the next chapter, we study generative adversarial networks, which approach generative modeling from a different perspective.

Guiding principle. Combining probabilistic modeling with neural networks enables flexible and scalable generative models.

Chapter 28

Generative Adversarial Networks

28.1 Adversarial Formulation

Generative adversarial networks (GANs) provide an alternative approach to generative modeling based on a game between two models.

Core idea. Two networks are trained simultaneously:

- **Generator** G_θ: maps noise $z \sim p(z)$ to data space
- **Discriminator** D_ϕ: distinguishes real data from generated data

Minimax objective.

$$\min_\theta \max_\phi \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D_\phi(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D_\phi(G_\theta(z)))].$$

Interpretation.

- The generator tries to fool the discriminator
- The discriminator tries to distinguish real and fake samples

Equilibrium. At optimality, the generator produces samples indistinguishable from real data.

Implicit modeling. GANs define a sampling procedure without explicitly modeling $p_\theta(x)$.

Insight. Learning is framed as a game between two competing objectives.

28.2 Divergence Minimization

GANs can be understood as minimizing a divergence between distributions.

Optimal discriminator. For a fixed generator, the optimal discriminator is:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_\theta(x)}.$$

Resulting objective. Substituting D^* yields:

$$\min_\theta \ \mathrm{JS}(p_{\text{data}} \,\|\, p_\theta),$$

where JS denotes the Jensen–Shannon divergence.

Interpretation.

- GAN training implicitly minimizes a divergence between distributions
- The generator improves as distributions become closer

Limitations.

- Divergences may behave poorly when supports do not overlap

Insight. GANs connect game-theoretic learning with statistical divergence minimization.

28.3 Training Instability

Training GANs is often challenging due to the adversarial nature of the objective.

Non-convex–non-concave optimization. The minimax problem does not generally admit stable convergence.

Mode collapse.

- Generator produces limited variety of outputs
- Fails to capture full data distribution

Oscillatory dynamics.

- Generator and discriminator may fail to converge
- Parameters can cycle or diverge

Gradient issues.

- Vanishing gradients when discriminator is too strong
- Unstable updates due to adversarial feedback

Practical techniques.

- Careful balancing of generator and discriminator updates
- Label smoothing
- Gradient penalties

Insight. Adversarial training introduces dynamics that are fundamentally different from standard optimization.

28.4 Variants and Improvements

Many variants of GANs have been proposed to address training challenges and improve performance.

Wasserstein GAN (WGAN).

- Replaces JS divergence with Wasserstein distance
- Improves stability and gradient behavior

WGAN with gradient penalty.

- Enforces Lipschitz constraint via gradient penalties

Conditional GANs.

- Generate samples conditioned on labels or inputs

CycleGAN and image translation.

- Learn mappings between domains without paired data

Style-based GANs.

- Introduce structured latent spaces
- Enable fine control over generated images

Insight. Improving GANs often involves modifying the objective, architecture, or training procedure.

28.5 A Unifying Perspective

GANs provide a distinct approach to generative modeling:

- **Implicit models:** define sampling without explicit density
- **Adversarial training:** two networks in competition
- **Divergence minimization:** alignment of distributions

Core components.

- Generator G_θ
- Discriminator D_ϕ
- Minimax objective

Key insight. Adversarial learning provides a powerful but challenging framework for generative modeling.

28.6 Outlook

This chapter introduced generative adversarial networks:

- adversarial formulation,
- divergence-based interpretation,
- training challenges,
- key variants and improvements.

In the next chapter, we study diffusion models, which provide a fundamentally different approach to generative modeling based on stochastic processes.

Guiding principle. Different generative frameworks offer different tradeoffs between tractability, stability, and sample quality.

Chapter 29

Normalizing Flows

29.1 Change of Variables Formula

Normalizing flows provide a class of generative models based on invertible transformations of probability distributions.

Core idea. Start with a simple base distribution $z \sim p(z)$ and transform it into a complex distribution $x = f_\theta(z)$.

Change of variables formula. If f is invertible and differentiable, then

$$p_\theta(x) = p(z) \left| \det \left(\frac{\partial z}{\partial x} \right) \right|, \quad \text{where } z = f^{-1}(x).$$

Equivalent form.

$$\log p_\theta(x) = \log p(z) + \log \left| \det \left(\frac{\partial z}{\partial x} \right) \right|.$$

Interpretation.

- Density changes according to local volume distortion
- The determinant measures expansion or contraction

Insight. Normalizing flows transform simple distributions into complex ones while keeping track of density changes.

29.2 Invertible Transformations

The design of normalizing flows relies on constructing invertible transformations that are both expressive and computationally efficient.

Requirements.

- f must be invertible
- The Jacobian determinant must be tractable

Composition of transformations.

$$x = f_L \circ f_{L-1} \circ \cdots \circ f_1(z).$$

Jacobian factorization.

$$\log \left| \det \frac{\partial z}{\partial x} \right| = \sum_{\ell=1}^{L} \log \left| \det \frac{\partial h_{\ell-1}}{\partial h_\ell} \right|.$$

Types of transformations.

- Affine coupling layers
- Autoregressive transformations
- Volume-preserving flows

Tradeoff.

- Expressivity vs computational efficiency

Insight. Careful design of invertible mappings enables tractable and expressive generative models.

29.3 Exact Likelihood Computation

A key advantage of normalizing flows is the ability to compute exact likelihoods.

Likelihood.

$$\log p_\theta(x) = \log p(f^{-1}(x)) + \log \left| \det \left(\frac{\partial f^{-1}}{\partial x} \right) \right|.$$

Comparison with other models.

- VAEs: approximate likelihood
- GANs: implicit likelihood
- Flows: exact likelihood

Sampling.

- Sample $z \sim p(z)$

- Compute $x = f_\theta(z)$

Inference.

- Compute $z = f_\theta^{-1}(x)$
- Evaluate density using change of variables

Efficiency considerations.

- Jacobian determinant must be efficiently computable
- Often achieved via triangular or structured matrices

Insight. Normalizing flows provide a rare combination of exact likelihood and efficient sampling.

29.4 Model Design

Designing effective normalizing flows involves balancing expressivity and tractability.

Coupling layers.

- Partition variables into two parts
- Transform one part conditioned on the other

Autoregressive flows.

- Model dependencies sequentially
- Triangular Jacobian simplifies determinant computation

Stacking transformations.

- Multiple layers increase expressivity
- Each layer contributes to transformation complexity

Invertible neural networks.

- Architectures designed to ensure invertibility

Limitations.

- Architectural constraints for invertibility
- Computational cost for high-dimensional data

Insight. Model design in flows requires balancing flexibility with computational constraints.

29.5 A Unifying Perspective

Normalizing flows combine several key ideas:

- **Transformations:** mapping simple distributions to complex ones
- **Invertibility:** enabling exact density computation
- **Composition:** building expressive models through layers

Core components.

- Base distribution $p(z)$
- Invertible mapping f_θ
- Jacobian determinant

Key insight. By carefully designing invertible transformations, we can build expressive generative models with exact likelihoods.

29.6 Outlook

This chapter introduced normalizing flows:

- change of variables for density transformation,
- invertible transformations,
- exact likelihood computation,
- practical model design.

In the next chapter, we study diffusion models, which approach generative modeling through stochastic processes and iterative refinement.

Guiding principle. Structured transformations of probability distributions provide a powerful framework for generative modeling.

Chapter 30

Diffusion Models: Intuition

30.1 Noise Injection Processes

Diffusion models are generative models based on gradually transforming data into noise and then reversing this process.

Forward process. We define a sequence of random variables $(x_0, x_1, \ldots, x_T)$ where:

- $x_0 \sim p_{\text{data}}$
- x_T approaches a simple distribution (e.g., Gaussian noise)

Noise injection. At each step, we add small Gaussian noise:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I).$$

Markov structure.

- Each step depends only on the previous state
- The process is a Markov chain

Effect.

- Gradually destroys structure in data
- Transforms data distribution into noise

Insight. The forward process defines a controlled way to map complex data distributions to simple ones.

30.2 Denoising Perspective

The generative process learns to reverse the noise injection.

Reverse process. We aim to model:

$$p_\theta(x_{t-1} \mid x_t),$$

which removes noise step by step.

Denoising interpretation.

- Each step predicts a less noisy version of the data
- The model learns to recover structure from noise

Learning objective. The model is trained to predict the noise added at each step:

$$\varepsilon_\theta(x_t, t) \approx \varepsilon_t.$$

Simplified loss.

$$\mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right].$$

Sampling.

- Start from noise $x_T \sim \mathcal{N}(0, I)$
- Iteratively denoise to obtain x_0

Insight. Generation is achieved by repeatedly denoising random noise into structured data.

30.3 Score-Based View

Diffusion models can also be interpreted through score functions.

Score function.

$$s(x) = \nabla_x \log p(x).$$

Key idea. Instead of modeling $p(x)$ directly, learn its score.

Connection to diffusion.

- The denoising function approximates the score of noisy distributions
- Each step corresponds to estimating $\nabla_x \log p_t(x)$

Langevin dynamics.

$$x_{t+1} = x_t + \eta \, \nabla_x \log p(x_t) + \sqrt{2\eta} \, \xi_t,$$

where ξ_t is Gaussian noise.

Interpretation.

- Samples are generated by following gradients of log-density
- Noise ensures exploration of the distribution

Insight. Diffusion models connect denoising with gradient-based sampling from probability distributions.

30.4 Conceptual Connections

Diffusion models unify several ideas from probability, optimization, and physics.

Connections to other models.

- **Latent variable models:** introduce intermediate noisy states
- **Energy-based models:** relate to score functions
- **Markov processes:** sequential transformations

Continuous-time limit.

- Discrete steps approximate stochastic differential equations
- Leads to a continuous diffusion process

Geometry of data.

- Noise smooths the data distribution
- Denoising recovers structure

Comparison with other generative models.

- VAEs: probabilistic latent modeling
- GANs: adversarial training
- Flows: invertible transformations
- Diffusion: iterative refinement

Insight. Diffusion models combine probabilistic modeling with iterative denoising and gradient-based sampling.

30.5 A Unifying Perspective

Diffusion models can be understood through multiple lenses:

- **Probabilistic:** modeling distributions through noise processes
- **Algorithmic:** iterative denoising procedures
- **Geometric:** following gradients in data space

Core components.

- Forward noise process
- Reverse denoising process
- Learned score or noise predictor

Key insight. Complex data distributions can be generated by reversing a simple stochastic process.

30.6 Outlook

This chapter introduced diffusion models from an intuitive perspective:

- noise injection processes,
- denoising as generation,
- score-based interpretations,
- conceptual connections to other models.

In the next chapter, we study diffusion models more rigorously, including their probabilistic formulation and training objectives.

Guiding principle. Complex structure can be learned by progressively refining simple noise through learned transformations.

Chapter 31

Diffusion Models: Mathematical Foundations

31.1 Forward and Reverse Processes

Diffusion models are built upon a pair of stochastic processes: a forward (noising) process and a reverse (denoising) process.

Forward process. We define a Markov chain $(x_0, x_1, \ldots, x_T)$ such that:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I),$$

where $\{\beta_t\}_{t=1}^T$ is a variance schedule.

Closed-form expression. By composing Gaussian transitions, we obtain:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t)I),$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

Interpretation.

- As t increases, x_t becomes increasingly noisy
- For large T, $x_T \approx \mathcal{N}(0, I)$

Reverse process. We seek to model:

$$p_\theta(x_{t-1} \mid x_t),$$

which is also Gaussian under suitable assumptions:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)).$$

Learning objective. The parameters θ are trained to approximate the true reverse transitions.

Insight. The forward process is fixed and simple, while the reverse process is learned.

31.2 Score Matching

An alternative formulation of diffusion models is based on score matching.

Score function.

$$s_t(x) = \nabla_x \log p_t(x),$$

where $p_t(x)$ is the distribution of x_t.

Objective. Instead of modeling $p_t(x)$ directly, we learn $s_t(x)$.

Denoising score matching. For Gaussian perturbations, the score satisfies:

$$s_t(x_t) = -\frac{1}{1-\bar{\alpha}_t}(x_t - \sqrt{\bar{\alpha}_t}x_0).$$

Learning formulation. We train a neural network $s_\theta(x_t, t)$ to approximate the score:

$$\mathbb{E}_{t,x_0,\varepsilon}\left[\|s_\theta(x_t,t) - \nabla_{x_t}\log q(x_t \mid x_0)\|^2\right].$$

Equivalence. This objective is equivalent to predicting the noise ε.

Insight. Score matching provides a principled way to learn gradients of log-densities.

31.3 Langevin Dynamics

Sampling from a distribution can be achieved using Langevin dynamics.

Langevin update.

$$x_{k+1} = x_k + \eta\,\nabla_x \log p(x_k) + \sqrt{2\eta}\,\xi_k, \quad \xi_k \sim \mathcal{N}(0, I).$$

Interpretation.

- Gradient term drives samples toward high-density regions
- Noise term ensures exploration

Connection to diffusion models.

- Learned score approximates $\nabla_x \log p(x)$
- Sampling uses score-based updates

Continuous-time limit. Langevin dynamics corresponds to a stochastic differential equation:

$$dx_t = \nabla_x \log p(x_t)\,dt + \sqrt{2}\,dW_t,$$

where W_t is Brownian motion.

Insight. Diffusion sampling can be viewed as approximate Langevin dynamics with learned scores.

31.4 Sampling Algorithms

Several sampling procedures arise from the mathematical formulation of diffusion models.

Ancestral sampling.

- Start from $x_T \sim \mathcal{N}(0, I)$
- Iteratively sample:
$$x_{t-1} \sim p_\theta(x_{t-1} \mid x_t)$$

Deterministic variants.

- Use deterministic updates instead of sampling
- Reduce variance and speed up generation

Score-based sampling.

- Use discretized Langevin dynamics
- Update using learned score function

Tradeoffs.

- More steps $\to$ higher quality samples
- Fewer steps $\to$ faster generation

Numerical considerations.

- Choice of step size
- Stability of updates

Insight. Sampling is an iterative refinement process guided by learned gradients.

31.5 A Unifying Perspective

Diffusion models can be understood through multiple mathematical frameworks:

- **Markov processes:** forward and reverse chains
- **Score matching:** learning gradients of log-density
- **Stochastic dynamics:** Langevin equations

Core components.

- Forward noise process $q(x_t \mid x_{t-1})$
- Reverse process $p_\theta(x_{t-1} \mid x_t)$
- Score function $s_\theta(x_t, t)$

Key insight. Diffusion models unify probabilistic modeling, stochastic processes, and gradient-based sampling.

31.6 Outlook

This chapter developed the mathematical foundations of diffusion models:

- forward and reverse stochastic processes,
- score matching formulations,
- Langevin dynamics,
- sampling algorithms.

In the next chapter, we study modern refinements and practical implementations of diffusion models.

Guiding principle. Complex distributions can be sampled by following learned gradients within a stochastic dynamical system.

Chapter 32

Large-Scale Generative Models

32.1 Diffusion in Practice (e.g., Stable Diffusion)

Diffusion models have become one of the dominant approaches to large-scale generative modeling, particularly in image synthesis.

From theory to practice. Practical diffusion systems extend the basic framework with architectural and algorithmic improvements:

- U-Net architectures for denoising
- Conditioning mechanisms (e.g., text embeddings)
- Efficient sampling strategies

Latent diffusion. Instead of operating directly in pixel space, models operate in a lower-dimensional latent space:

- Encode images into latent representations
- Perform diffusion in latent space
- Decode back to pixel space

Advantages.

- Reduced computational cost
- Improved scalability

Conditioning. Generative models can be guided by additional inputs:

- Text prompts
- Class labels
- Other modalities

Insight. Modern diffusion systems combine probabilistic modeling with powerful neural architectures and conditioning mechanisms.

32.2 Large Language Models

Large language models (LLMs) are generative models for sequences, typically based on transformer architectures.

Autoregressive modeling. LLMs model the joint distribution of a sequence:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}).$$

Training objective.

$$\max_{\theta} \mathbb{E}\left[\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})\right].$$

Transformer backbone.

- Self-attention layers
- Feedforward networks
- Residual connections and normalization

Capabilities.

- Text generation
- Question answering
- Code generation
- Reasoning tasks

Pretraining and fine-tuning.

- Pretrained on large corpora
- Adapted to downstream tasks

Insight. LLMs leverage scale and architecture to model complex sequential dependencies.

32.3 Scaling Laws

Empirical studies reveal systematic relationships between model performance and scale.

Scaling factors.

- Model size (number of parameters)
- Dataset size
- Compute resources

Empirical law.

$$\text{Error} \propto N^{-\alpha},$$

where N represents scale and $\alpha > 0$.

Implications.

- Larger models tend to perform better
- Gains diminish with scale

Compute-optimal tradeoffs.

- Balance model size and dataset size
- Efficient allocation of computational resources

Emergent behavior.

- New capabilities appear at large scale
- Not easily predicted from small models

Insight. Scaling is a fundamental driver of performance in modern machine learning systems.

32.4 Engineering Considerations

Large-scale generative models require careful engineering to train and deploy.

Distributed training.

- Data parallelism
- Model parallelism
- Pipeline parallelism

Hardware acceleration.

- GPUs and specialized accelerators
- Memory and bandwidth constraints

Optimization challenges.

- Large batch sizes
- Learning rate scheduling
- Stability at scale

Inference considerations.

- Latency vs throughput
- Model compression and quantization

Data considerations.

- Data quality and diversity
- Preprocessing and filtering

Safety and alignment.

- Controlling model outputs
- Mitigating harmful or biased behavior

Insight. Engineering plays a crucial role in transforming theoretical models into practical systems.

32.5 A Unifying Perspective

Large-scale generative models integrate multiple dimensions:

- **Modeling:** probabilistic and neural architectures
- **Computation:** large-scale optimization
- **Data:** massive and diverse datasets
- **Systems:** distributed and efficient implementations

Core idea. Performance emerges from the interaction between models, data, and computation.

Key insight. Modern generative models are not defined solely by their mathematical formulation, but by their scale and system-level design.

32.6 Outlook

This chapter examined large-scale generative models:

- diffusion models in practice,
- large language models,
- scaling laws,
- engineering considerations.

This concludes our study of modern generative modeling techniques.

Guiding principle. Advances in machine learning arise from the combination of theory, architecture, data, and computation at scale.

Chapter 33

Generalization in Deep Learning

33.1 Overparameterization

Classical learning theory suggests that models with too many parameters should overfit. However, modern deep learning often operates in highly overparameterized regimes, where the number of parameters exceeds the number of training samples.

Observation.

- Deep networks can fit training data exactly
- Yet they often generalize well to unseen data

Interpolation regime. In overparameterized models, it is often possible to achieve zero training error:

$$\hat{R}_n(\theta) = 0.$$

Multiple solutions. There are many parameter configurations that interpolate the data.

Key question. Why do some of these solutions generalize well while others do not?

Effective complexity.

- Parameter count alone does not determine generalization
- Structure and training dynamics play a role

Insight. Overparameterization changes the nature of learning: instead of selecting among models, we select among solutions.

33.2 Implicit Bias of Gradient Methods

Optimization algorithms do not explore all possible solutions equally. Instead, they exhibit implicit bias toward certain solutions.

Implicit bias. The tendency of an optimization method to favor particular solutions among many that achieve low training error.

Example (linear regression). In underdetermined systems, gradient descent converges to the minimum-norm solution:

$$\min \|w\|_2 \quad \text{subject to } Xw = y.$$

Deep networks.

- Implicit bias is more complex
- Depends on architecture, initialization, and optimizer

Role of SGD.

- Noise in stochastic gradients influences trajectories
- May favor flatter minima

Flat vs sharp minima.

- Flat minima: more robust to perturbations
- Sharp minima: more sensitive, potentially worse generalization

Insight. Generalization is shaped not only by the model, but by how it is optimized.

33.3 Double Descent Phenomenon

Classical bias–variance tradeoff predicts that increasing model complexity leads to overfitting. However, modern observations reveal a more nuanced behavior.

Double descent.

- Test error first decreases, then increases, and then decreases again as model size grows

Phases.

- Underparameterized regime: classical behavior
- Interpolation threshold: peak test error

- Overparameterized regime: improved generalization

Interpretation.

- Larger models can find better interpolating solutions
- Optimization plays a key role

Empirical evidence.

- Observed in neural networks, kernel methods, and other models

Insight. Model complexity and generalization have a richer relationship than classical theory suggests.

33.4 Open Questions

Despite significant progress, the theory of generalization in deep learning remains incomplete.

Key challenges.

- Understanding why overparameterized models generalize
- Characterizing implicit bias in deep networks
- Explaining the role of optimization dynamics

Theoretical directions.

- Complexity measures beyond parameter count
- Stability-based generalization bounds
- Information-theoretic approaches

Connections to other fields.

- Statistical physics (energy landscapes)
- Dynamical systems (training dynamics)
- Information theory (compression and representation)

Practical implications.

- Design of architectures
- Choice of optimization algorithms
- Regularization strategies

Insight. Understanding generalization in deep learning is an active area of research, bridging theory and practice.

33.5 A Unifying Perspective

Generalization in deep learning arises from the interaction of multiple factors:

- **Model capacity:** expressivity of neural networks
- **Optimization:** implicit bias of training algorithms
- **Data:** structure and distribution of inputs

Core idea. Generalization is not determined by a single factor, but by the dynamics of the entire learning process.

Key insight. Modern deep learning challenges classical intuitions, requiring new theoretical frameworks.

33.6 Outlook

This chapter examined generalization in deep learning:

- overparameterization and interpolation,
- implicit bias of optimization methods,
- double descent phenomenon,
- open theoretical questions.

This concludes our exploration of the theoretical and practical foundations of modern machine learning and deep learning.

Guiding principle. Understanding generalization requires integrating ideas from optimization, probability, and representation learning.

Chapter 34

Geometry and Dynamics of Learning

34.1 Loss Landscapes

Training a neural network can be viewed as navigating a high-dimensional loss landscape.

Loss function. Given parameters $\theta \in \mathbb{R}^d$, the loss defines a function:

$$L(\theta) : \mathbb{R}^d \to \mathbb{R}.$$

Landscape interpretation.

- Points correspond to parameter configurations
- Height corresponds to loss value

Critical points.

- Local minima: $\nabla L(\theta) = 0$, positive curvature
- Saddle points: mixed curvature directions

High-dimensional geometry.

- Saddle points are more prevalent than poor local minima
- Many directions are nearly flat

Flat vs sharp minima.

- Flat minima: robust to perturbations

- Sharp minima: sensitive to parameter changes

Visualization (conceptual). Low-dimensional projections reveal valleys, ridges, and plateaus.

Insight. The geometry of the loss landscape strongly influences optimization and generalization.

34.2 Neural Tangent Kernel (Intuition)

The neural tangent kernel (NTK) provides a linearized view of neural network training.

Linearization. Around initialization θ_0, the network can be approximated as:

$$f_\theta(x) \approx f_{\theta_0}(x) + \nabla_\theta f_{\theta_0}(x)^\top (\theta - \theta_0).$$

Neural tangent kernel.

$$K(x, x') = \nabla_\theta f_{\theta_0}(x)^\top \nabla_\theta f_{\theta_0}(x').$$

Interpretation.

- Defines a kernel induced by the network at initialization
- Training resembles kernel regression with kernel K

Infinite-width limit.

- NTK becomes deterministic
- Training dynamics become linear

Insight. The NTK connects deep learning with classical kernel methods.

34.3 Continuous-Time View of Training

Gradient-based optimization can be viewed as a continuous-time dynamical system.

Gradient descent update.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t).$$

Continuous-time limit. As $\eta \to 0$, we obtain a differential equation:

$$\frac{d\theta(t)}{dt} = -\nabla L(\theta(t)).$$

Gradient flow.

- Parameters evolve along the steepest descent direction
- Trajectories follow curves in parameter space

Energy interpretation. The loss decreases along trajectories:

$$\frac{d}{dt}L(\theta(t)) = -\|\nabla L(\theta(t))\|^2 \leq 0.$$

Extension to stochastic dynamics. Stochastic gradient descent introduces noise:

$$d\theta(t) = -\nabla L(\theta(t))dt + \Sigma^{1/2}dW_t,$$

where W_t is Brownian motion.

Insight. Training can be understood as a continuous flow shaped by both gradients and noise.

34.4 Dynamical Systems Perspective

Neural network training and representation learning can be viewed through the lens of dynamical systems.

Parameter dynamics.

- Evolution of $\theta(t)$ under optimization

Representation dynamics.

- Evolution of features $f_{\theta(t)}(x)$ during training

Stability.

- Stable fixed points correspond to minima
- Stability depends on curvature and noise

Attractors.

- Optimization may converge to particular regions of parameter space

Connection to residual networks. Deep networks resemble discretized dynamical systems.

Feature evolution.

- Early training: rapid feature changes
- Later stages: fine adjustments

Insight. Learning is a dynamic process that evolves both parameters and representations over time.

34.5 A Unifying Perspective

The geometry and dynamics of learning provide a unified framework:

- **Geometry:** structure of the loss landscape
- **Dynamics:** trajectories of optimization
- **Linearization:** NTK approximation
- **Stochasticity:** role of noise in training

Core idea. Understanding learning requires analyzing both the structure of the objective and the dynamics of optimization.

Key insight. Modern deep learning lies at the intersection of geometry, probability, and dynamical systems.

34.6 Outlook

This chapter examined the geometry and dynamics of learning:

- loss landscapes and their structure,
- neural tangent kernel perspective,
- continuous-time view of optimization,
- dynamical systems interpretation.

These perspectives provide a deeper theoretical understanding of how learning occurs in modern machine learning systems.

Guiding principle. Learning is both a geometric problem and a dynamical process, shaped by the interplay between model structure and optimization.

Chapter 35

Practical System Design

35.1 Data Pipelines

In modern machine learning systems, data is as important as models. Effective system design begins with reliable and scalable data pipelines.

Data pipeline. A data pipeline transforms raw data into a form suitable for training and evaluation.

Stages.

- Data collection
- Cleaning and preprocessing
- Feature extraction or encoding
- Storage and access

Preprocessing.

- Normalization and scaling
- Handling missing values
- Data augmentation

Streaming vs batch processing.

- Batch pipelines process data in large chunks
- Streaming pipelines process data continuously

Data quality.

- Consistency and correctness

- Noise and bias considerations

Insight. The quality of a machine learning system is fundamentally limited by the quality of its data.

35.2 Training Workflows

Training modern models requires coordinated workflows that manage data, computation, and experimentation.

Training loop.

1. Load batch of data
2. Compute predictions
3. Evaluate loss
4. Compute gradients
5. Update parameters

Experiment management.

- Track hyperparameters
- Record metrics and logs
- Compare different model configurations

Hyperparameter tuning.

- Grid search, random search
- Adaptive methods (e.g., Bayesian optimization)

Distributed training.

- Data parallelism across multiple devices
- Model parallelism for large models

Checkpointing.

- Save intermediate model states
- Enable recovery and experimentation

Insight. Well-designed workflows enable efficient experimentation and reliable model development.

35.3 Debugging and Diagnostics

Training machine learning models often involves diagnosing issues that arise during optimization.

Common problems.

- Training does not converge
- Overfitting or underfitting
- Numerical instability

Diagnostic tools.

- Training and validation curves
- Gradient norms
- Activation statistics

Sanity checks.

- Verify loss decreases on small datasets
- Check gradient correctness
- Ensure labels and data are consistent

Visualization.

- Monitor metrics over time
- Inspect intermediate outputs

Systematic debugging.

- Isolate components
- Simplify the model
- Incrementally increase complexity

Insight. Debugging is an essential skill in machine learning, requiring both theoretical understanding and practical experience.

35.4 Reproducibility

Reproducibility is critical for scientific and engineering reliability.

Definition. A result is reproducible if it can be consistently obtained under the same conditions.

Sources of variability.

- Random initialization
- Data shuffling
- Hardware and numerical differences

Best practices.

- Fix random seeds
- Record configurations and hyperparameters
- Version datasets and code

Experiment tracking.

- Store results systematically
- Maintain logs of experiments

Determinism vs performance.

- Fully deterministic execution may reduce performance
- Tradeoff between reproducibility and efficiency

Insight. Reproducibility ensures that results are reliable and can be built upon.

35.5 A Unifying Perspective

Practical system design integrates multiple components:

- **Data:** pipelines and preprocessing
- **Computation:** training workflows
- **Diagnostics:** debugging tools
- **Reliability:** reproducibility practices

Core idea. Successful machine learning systems require careful coordination of data, models, and infrastructure.

Key insight. Practical effectiveness depends not only on algorithms, but on the systems that support them.

35.6 Outlook

This chapter examined practical aspects of machine learning systems:

- data pipelines,
- training workflows,
- debugging and diagnostics,
- reproducibility.

This concludes the book.

Final guiding principle. Machine learning is both a mathematical discipline and an engineering practice, requiring insight into theory, data, and systems.

Chapter 36

Limits and Future Directions

36.1 Robustness and Uncertainty

Modern machine learning systems achieve impressive performance, yet they remain sensitive to perturbations and uncertainty.

Robustness. A model is robust if its predictions remain stable under small changes to input:

$$\|x - x'\| \text{ small } \Rightarrow \; f(x) \approx f(x').$$

Adversarial examples.

- Carefully crafted perturbations can significantly alter predictions
- Often imperceptible to humans

Distribution shift.

- Training and test distributions may differ
- Performance can degrade under new conditions

Uncertainty estimation.

- **Aleatoric uncertainty:** inherent noise in data
- **Epistemic uncertainty:** uncertainty due to limited knowledge

Approaches.

- Bayesian methods

- Ensemble models
- Calibration techniques

Insight. Reliable AI systems must quantify and manage uncertainty in addition to making accurate predictions.

36.2 Failure Modes of Modern AI

Despite strong empirical performance, modern models exhibit systematic failure modes.

Common issues.

- Overconfidence in incorrect predictions
- Sensitivity to spurious correlations
- Lack of robustness to rare or out-of-distribution inputs

Data dependence.

- Models inherit biases present in training data
- Performance reflects data distribution rather than true understanding

Generalization limits.

- Difficulty extrapolating beyond training regime
- Challenges with compositional reasoning

Scaling limitations.

- Increasing size does not solve all problems
- Some failures persist even at large scale

Insight. Understanding failure modes is essential for improving reliability and guiding future research.

36.3 Interpretability

Interpretability seeks to understand how and why models make predictions.

Motivation.

- Trust and transparency
- Debugging and model improvement

- Scientific understanding

Types of interpretability.

- **Feature attribution:** importance of inputs
- **Representation analysis:** structure of learned features
- **Model simplification:** approximating complex models with simpler ones

Challenges.

- High-dimensional representations
- Nonlinear interactions
- Lack of unique explanations

Tradeoffs.

- Accuracy vs interpretability
- Simplicity vs fidelity

Insight. Interpretability is not a single property, but a collection of approaches to understanding complex models.

36.4 Future Research Directions

The future of machine learning lies at the intersection of theory, data, and systems.

Theoretical directions.

- Understanding generalization in deep learning
- Characterizing optimization dynamics
- Bridging discrete and continuous models

Modeling directions.

- Multimodal models combining vision, language, and action
- More efficient and scalable architectures
- Models with improved reasoning capabilities

Data and learning paradigms.

- Self-supervised and unsupervised learning

- Learning from limited or noisy data
- Continual and lifelong learning

Systems and efficiency.

- Reducing computational and energy costs
- Efficient training and inference

Safety and alignment.

- Ensuring reliable and controllable behavior
- Mitigating unintended consequences

Interdisciplinary connections.

- Neuroscience and cognitive science
- Physics and dynamical systems
- Mathematics and statistics

Insight. Future progress will require integrating insights across disciplines and balancing theory with empirical advances.

36.5 A Final Perspective

This book has explored machine learning and deep learning from both mathematical and practical perspectives.

Core themes.

- Learning as function approximation and optimization
- Representation learning through deep architectures
- Probabilistic modeling and generative processes
- Scaling and system-level considerations

Unifying idea. Learning systems are shaped by the interaction of data, models, and algorithms.

Final insight. Artificial intelligence is an evolving field, where theory and practice continuously inform and challenge each other.

36.6 Closing Remarks

Machine learning stands at a unique intersection of disciplines, combining rigorous mathematics with practical engineering.

Outlook.

- New theoretical frameworks will deepen our understanding
- New architectures and systems will expand capabilities
- Responsible development will shape real-world impact

Final guiding principle. The future of AI depends not only on advancing performance, but on understanding, reliability, and responsible use.